

Two-Tier Black-box Blockchains and Application to Instant Layer-1 Payments

Michele Ciampi¹, Yun Lu², Rafail Ostrovsky³, and Vassilis Zikas⁴

¹ University of Edinburgh
michele.ciampi@ed.ac.uk

² University of Victoria
yunlu@uvic.ca

³ UCLA
rafail@cs.ucla.edu

⁴ Georgia Tech
vzikas@gatech.edu

Abstract. Common blockchain protocols are *monolithic*, i.e., their security relies on a single assumption, e.g., honest majority of hashing power (Bitcoin) or stake (Cardano, Algorand, Ethereum). In contrast, so-called optimistic approaches (Thunderella, Meshcash) rely on a combination of assumptions to achieve faster transaction liveness.

We revisit, redesign, and augment the optimistic paradigm to a tiered approach. Our design assumes a primary (Tier 1) and a secondary (Tier 2, also referred to as *fallback*) blockchain, and achieves full security also in a tiered fashion: If the assumption underpinning the primary chain holds, then we guarantee safety, liveness and censorship resistance, irrespectively of the status of the fallback chain. And even if the primary assumption fails, all security properties are still satisfied (albeit with a temporary slow down) provided the fallback assumption holds. To our knowledge, no existing optimistic or tiered approach preserves both safety and liveness when any one of its underlying blockchain (assumptions) fails. The above is achieved by a new detection-and-recovery mechanism that links the two blockchains, so that any violation of safety, liveness, or censorship resistance on the (faster) primary blockchain is temporary—it is swiftly detected and recovered on the secondary chain—and thus cannot result in a persistent fork or halt of the blockchain ledger.

We instantiate the above paradigm using a primary chain based on *proof of reputation* (PoR) and a fallback chain based on *proof of stake* (PoS). Our construction uses the PoR and PoS blockchains in a mostly black-box manner—where rather than assuming a concrete construction we distil abstract properties on the two blockchains that are sufficient for applying our tiered methodology. In fact, choosing reputation as the resource of the primary chain opens the door to an incentive mechanism—which we devise and analyze—that tokenizes reputation in order to deter cheating and boost participation (on both the primary/PoR and the fallback/PoS blockchain). As we demonstrate, such tokenization in combination with interpreting reputation as a built-in system-wide credit score, allows for embedding in our two-tiered methodology a novel mechanism which provides collateral-free, multi-use payment-channel-like functionality where payments can be instantly confirmed.

1 Introduction

The security of distributed cryptographic protocols typically relies on (unverifiable) assumptions about the number of (or the resources held by) corrupted vs. uncorrupted (aka honest) parties. As an example, it is known that an honest majority is necessary and sufficient for secure decentralization in the synchronous model; this includes (blockchain-based) decentralized ledgers, where honest majority (of either parties or a resource like stake, e.g., [Eth] or hashing-power [Sat08]) is required. In a blockchain-based cryptocurrency, which might hold billions of dollars, violation of such assumptions can have devastating effect for its users, e.g., [ETC, Ver].

Two types of defenses are typically employed to fortify blockchains in face of such silent threats: The first one is a game-theoretic argument often used to argue the so-called *economic robustness* of primitives, i.e., their resistance to rational attackers that might violate the underlying cryptographic assumption, but are only willing to perform attacks that maximally increase their utility. Such arguments have been used in the cryptography research literature [GKM⁺13, BGM⁺18, BLZ21, AAMM24, AST24, RAKM23].

but have also become common in mainstream cryptocurrencies, like (PoS-based) Ethereum [Eth]. The second is to use *fallback security* where the security assumption is fortified by an additional assumption, so that only if both fail the protocol might lose (all) its security properties. This fallback paradigm has been employed in a wide range of cryptographic constructions [KZZ16,BK14,MBB⁺19,AKMZ12,CLZ22].

An advantage of fallback security is that one is able to use one of the assumptions in an optimistic manner. A classical example here is that of fairness in multiparty computation (MPC), e.g., [ASW97,ASW98]: It is well known that fair MPC requires that at most $t < n/2$ of the parties might be corrupted; however, without fairness, an arbitrary number, $t < n$ of corrupted parties can be tolerated (assuming a public-key infrastructure, in short PKI). Thus one can employ an external fall-back mechanism, e.g., a judge [ASW97] or a blockchain compensation mechanism [KZZ16,BK14], where we can optimistically run the (unfair) MPC protocol for $t < n$ and if a fairness violation is detected, then use the fallback to resolve the fairness violation. Such an optimistic model of execution has been also used in the blockchain realm to allow for better scalability and or network tolerance properties, with Thunderella [PS18] and Meshcash [BHMN21] being the most representative examples: In a nutshell, these protocols rely on smart refinements of the proof-of-work and/or proof-of-space-time paradigms, using novel optimistic methods to accelerate the blockchain and improve scalability and finality under more restrictive security assumption.

The starting point of our work is a novel, tiered fallback-mode for blockchains. The idea is to use two assumptions, each of which is sufficient for a blockchain, in a tiered manner: If *Assumption 1* (which is often called the *primary* assumption) is true (we refer to this case as the system’s *normal mode*) then the system maintains all its security and performance guarantees—including safety and liveness—irrespective of whether or not *Assumption 2* is true. Additionally, if *Assumption 1* fails, then the core properties of the system—safety and liveness—are not violated, but the system might be temporarily slowed down (we say that liveness gets slower). We refer to this latter case as *recovery mode*. We note that being able to guarantee both safety and liveness when either of the two assumptions fails, set our construction apart from the above optimistic approaches to blockchain [PS18,BHMN21]. In addition to achieving the above tiered security, as we show below, by instantiating our proposed construction with appropriate assumptions can combines the optimistic mode with rational arguments in order to achieve better liveness properties while embedding DeFi operations on layer one.⁵

We instantiate the above tiered paradigm as follows: For our primary chain we use reputation as a resource, relying on the Proof-of-Reputation (PoR) paradigm. We follow [KOZ20] for defining the relevant assumption (Assumption 1 in the above exposition); intuitively, reputation is utilized similar to how stake is used in a proof-of-stake (PoS) system, i.e., it is extracted from the contents of (a sufficiently deep block) in the blockchain⁶. In a nutshell, the reputation of a party corresponds to its probability of behaving honestly in the system. Our protocol uses these probabilities to select parties that are tasked to maintain the blockchain—in our iterated Byzantine fault-tolerant instantiation. These parties are block proposers and endorses for each slot (see below for more details). Then the primary assumption, hereafter referred to as the *PoR-assumption*, corresponds to requiring that the extracted reputations are close to the actual “ground-truth” probabilities of honest behavior (cf. Section 4.1.)

For our fallback chain, we use the *PoS assumption* that underpins the security of the PoS chain, e.g., that the majority of the stake in the system being held by honest parties⁷. We remark that, as discussed below, our results use mostly black-box access on the secondary (fall-back) chain, which can therefore be instantiated with different distributed ledgers—its fallback security will then rely on the security of that instantiation. However, for clarity, we chose to anchor our description to a PoS fallback blockchain.

The above combination of PoR with PoS fallback gives full security, i.e., both safety and liveness if the PoR assumption is true (independently of the stake distribution), and, if the PoR assumption fails but the PoS assumption is true, still both safety and liveness are guaranteed, albeit with a temporarily slower liveness parameter. More exciting, as discussed below, we introduce a mechanism which allows receivers to *instantly* confirm payments when the sender pledges a one-time collateral to them in case of a fork/double spending. With the use of a reputation system in PoR, we can even construct a mechanism

⁵ As we discuss below, such DeFi-style properties are also put on hold during *recovery mode*.

⁶ The choice of the concrete algorithm that extracts the reputation is orthogonal to our results and is therefore treated here as a black box.

⁷ As is common in the blockchain literature, we use the term “honest” to refer to parties that faithfully follow the protocol; unlike misbehavior, e.g., signing conflicting messages, honesty is not a property we can verify and is used only in the analysis of the protocol.

that does this *without* parties locking their own coins. This latter improvement can be achieved by interpreting reputation as the blockchain-analogue of credit-score, a paradigm which we believe can we use to embed also additional (classical) finance operation onto the above layer one thus opening the path to better DeFi applications, that leverage the above tiered fallback approach. In the following we give additional details on our contributions and techniques.

2 Our Contribution and Techniques

At a high level, our PoR-with-PoS-fallback design follows the PoR/PoS-hybrid blockchain blueprint of [KOZ20] (which shares several ideas with [PS18] and [BHMN21].) However, our approach includes multiple improvements over [KOZ20] and extensions—among others, a recovery mechanism when PoR fails, censorship resistance, concrete instantiation of the lottery for selecting minters, and explicit mechanisms (and economic robustness arguments) for participation and embedded DeFi operations. Furthermore, rather than anchoring our protocol to that of [KOZ20], we follow a more abstract, nearly black-box approach: We extract properties of the primary (PoR) and fallback (PoS) blockchains that are sufficient for our arguments; then any instantiation of the above chains which satisfies these properties would yield an instantiation of our paradigm.

In more detail, both the PoR and the PoS chains rely on a lottery for selecting the nodes that are in-charge of maintaining them. For the sake of exposition, let us focus on the following structure for both the primary (PoR) and fallback (PoS) ledger protocols: (1) The protocols advance in fully synchronized slots, where each slot might take several (synchronous) rounds and is associated with the creation of a specific block (the round and slot number is reflected in the block’s contents). (2) The lottery (which is seeded by randomness extracted from the PoR chain) selects *block proposers* and *block endorsers*: block proposers report their valid transactions they have heard and which are not already included in the previous slot to the block endorsers via a byzantine fault-tolerant broadcast protocol; the latter collect all valid transactions they receive (as broadcasted) from the proposers, add their signatures to it and propagate it (via standard gossip style diffusion) to the whole system. We stress that the above structure is for keeping the exposition as simple as possible, (and in particular will allow us to argue security assuming both sub-chains has finality); however, the proposed combination paradigm can be instantiated with different types of blockchains, including Nakamoto-style blockchains (although the concrete liveness and safety parameter will need to be tuned to account for eventual finality).

We stress that our fallback design assumes that a node runs both the PoR and the PoS chain (in parallel; however, in normal mode, the fallback chain has minimal transactions); in other words, the (Layer 1 of) the blockchain consists of a combination of the two ledgers. This is in contrast to approaches that use only PoS as Layer 1 and overlay a reputation-based protocol as a Layer 2 protocol to improve upon the properties of Layer 1 [BFK17,MMNT19]. We note in passing that such a Layer 2 solutions cannot achieve the fallback guarantees that we are aiming for, as failure of the assumption supporting Layer 1 would induce both safety and liveness failure, irrespective of how well the reputation reflects the ground truth.

We next sketch the main properties of our tiered blockchain design and the techniques for achieving them.

Detection and recovery from safety violation on the PoR chain. As already discussed in [KOZ20], reputation is a subjective and not necessarily future-proof resource. In particular, some malicious (covertly operating) nodes might behave in a way that makes the reputation-extraction algorithm assign to them high-reputation, and then, if they manage to form a majority of slot endorsers for some slot, attack the system, e.g., make it fork or stall. As discussed below, we will employ incentives, by means of reputation penalty, to deter such behavior, but this is not enough; we need a mechanism to ensure that such safety and liveness violations do not occur in the first place. For this we use the fallback PoS chain: the nodes that are selected in the PoS lottery to propose the next PoS block report their view of the most headers (these are hash pointers) of the PoR chain, starting from the hash-pointer reported in the previous PoS block. If any user observes that the reported hash-pointers are different than what he observes in his view of the PoR chain, then he complains in the next PoS block, also presenting evidence that his complain is valid—i.e., presenting the inputs to the contested hash chain. If such a dispute occurs, and both involved hash chains have *proper justification*, i.e., they are both

valid continuations of the PoR chain⁸ then the system enters a recovery mode, where the PoR chain takes control and resolves any double spending that might have occurred as a result of the above fork. As proven in [KOZ20], this can only happen when the assumption of the primary chain fails, i.e., the reputation considerably deviates from the ground truth meaning that the PoR assumption has failed. We also note that in the case a fork is detected on the fallback chain, there is sufficient ‘evidence’ to allow us to remove (i.e., zero out the reputation of) parties who have misbehaved, i.e., achieve a certain level of *accountable safety* (e.g., [GCN⁺25,NTT22,LPR25,NTT24]), by identifying at least one corrupt party in case of a safety violation.

Although the idea of a detection mechanism was already proposed in [KOZ20], there was no recovery mechanism, and the treatment considered the stake on the PoS system independent of the contents of the PoR. This, for example, could work if one uses an already deployed PoS blockchain as a fallback chain, where the fallback assumption is on the stake of that blockchain. In this work, we are using PoR and PoS to build a self-contained, Layer-1 blockchain; this means that both the stake and the reputation need to be extracted from the state of the system, which raises several technical challenges as discussed below. The first challenge is:

Where do we extract the reputation (resp. stake distribution) and seeds for the PoR (resp. PoS) lottery from?

The answer to the above question is not straightforward, because, due to the tiered security, the PoS chain is needed when PoR is found to be unreliable, e.g., it has forked. Hence one needs to be careful where to extract the reputation, stake distribution, and PoR and PoS lottery-seeds from—in particular, because the reputation is needed to check that a dispute on the PoS chain is justified. This is resolved by designating certain PoR-blockchain blocks (in predefined slot numbers) as *checkpointing* blocks and using them to extract the above runtime parameters. Such blocks need to be deep enough in the PoR chain, so that they have been already certified in the PoS chain without any dispute, and sufficient time has passed to ensure that any dispute should have been raised. By appropriate bounds on the network latency and the liveness parameters of the PoR vs PoS chains, we can provide such a depth.

The second challenge is with respect to recovery: Once the joint state of the two chains reflect the fact that the primary chain has forked, i.e., there is a dispute on the secondary chain with proper justification, the challenge becomes to recover from this situation without any honest user losing any funds. Since transactions require a signature from the owner of the funds, the only events that make such loss possible is double spending. To ensure that this event does not occur we require that a party only considers a transaction, say from PoR block B_i , as settled once the PoS block that endorses B_i is deep enough in the PoS chain and remains undisputed. Once again, this interval depends on the relative liveness of the PoR vs the PoS chain. (As we shall see below, our DeFi-style mechanisms also allows also for transactions can be confirmed at the speed of just one PoR slot).

Thus the core task of the recovery mechanism is to re-bootstrap the PoR chain. In a nutshell, this is done (in recovery mode) by temporarily using the PoS chain as primary, to create a new PoR genesis block, which includes (1) a pointer to the most recent PoR block B^* that has been certified in a deep-enough PoS block and is undisputed, (2) any transactions that were included (in any PoR-chain fork) following B^* —the PoS chain receives such transactions for a fixed period, and (3) any new transactions received (on the PoS chain) during recovery mode. Any parties who have been detected to sign conflicting messages have their reputation zeroed-out, which allows to sanitize the reputation as part of the recovery process.⁹ At the end of the recovery period, the PoS chain puts all the collected transaction in a PoR block that points (by a hash pointer) to B^* ; this new block along with the state extracted from the PoR chain up to B^* serve as a genesis block for restarting (in fact, resuming from the state ending to B^*) the PoR chain as the primary chain. We stress that the above is a very high level description, and a careful treatment is necessary to ensure security of the recovery procedure. The details can be found in Section 6.

⁸ Intuitively, “proper justification” includes evidence that the hash pointers were (part of blocks) that were signed by the majority of valid endorsers—i.e., endorsers that according to the PoR lottery were chosen for the corresponding slot.

⁹ This elimination of cheaters already allows for sanitizing; however, one can also use the recovery period to collect, on the PoS chain, new/additional/modified reputation data from the users and apply the reputation extraction AI on these new data.

Detection and recovery from liveness violation (on the PoR chain) and transaction censorship. The next new feature of our two-tier architecture is the detection and recovery of liveness violation and transaction censorship on the PoR chain. Let us start with liveness violation: The aforementioned safety-violation detection kicks in (on the PoS chain) when two parties report a fork, each with proper justification, as above.

However, a liveness violation (i.e., a party *not* sending something at the correct time) is much harder to detect and prove. In particular, it is shown that *accountable liveness* (analogous to the accountable safety discussed above) is impossible in our setting (of dishonest majority) [LPNRZ25]. Nevertheless, we can achieve a compromise that is still, in a sense, the “best of both worlds”: even if one of the PoR or PoS chains can fail, we either have liveness, or all honest parties will go into recovery mode, where reputations can be reset (e.g., through an external process). Intuitively, we do so by having PoS block creators vote on whether they believe a liveness violation is happening (this is abstracted through the external signal defined in Prop. 5.1). If PoR liveness is violated (and thus PoS assumptions hold), it means that (most) PoS blocks will have honest majority block creators—who will either (1) help propagate PoR blocks (if a PoR block is simply delayed), or (2) vote to confirm a liveness violation. In the case of (1), we still achieve liveness (albeit slightly slower). In case PoS block creators agree (have majority vote) on a liveness violation, then those disagreeing with the claim of liveness violation can post the contested PoR block (achieving liveness, again), or if no one disagrees, then this is evidence for liveness violation (i.e., no party claims to have the PoR block), and all parties can go into recovery mode to reset reputations. On the other hand, If PoR liveness is *not* violated, all honest parties would receive blocks without delay, and would never go into recovery regardless of the results of votes on the PoS.

Another complicated case is transaction censorship, aka transaction-liveness violation: A user u attempts to put its transaction to the PoR chain for a long time, but (due to violation of the PoR assumption) it is ignored by several consecutive slot committees. In this case, u posts its transaction (along with an annotation that this is a censored transaction) on the PoS blockchain—again, u by observing censorship, u knows that the PoR assumption failed, which means that the PoS assumption holds. Every party that observe such a transaction on the PoS chain, attempts to put it in the PoR chain in one of the next several blocks. If parties observe that an (allegedly) censored transaction fails to make it in the PoR chain by a sufficiently long timeout (cf. Section 6) they confirm the censorship (and therefore the issue on PoR) and can therefore trigger the recovery procedure as above. In this case, members of the most recent committee who did not include the transaction, have their reputation zeroed out. This does not penalize honest nodes, who would have seen the transaction on the PoS chain and would have therefore included it.

Reputation as DeFi-enabler: Instantly confirmed *safe transactions* Last but not least, we demonstrate how having (monetized, as above) reputation baked into the protocol’s Layer 1, allows for improving both the speed and the functionality of the blockchain system. In brief, the idea is the following: a party p_i can pledge (deposit/lock as collateral) a certain amount of coins v to a specific recipient p_j through a special PLEDGE transaction. Having created such a pledge, whenever p_i sends a transaction tx to p_j with amounts up to v , this transaction tx is considered *safe* and p_j can accept it as soon as they see it on a PoR block. Note that since p_j did not wait for tx to settle on the secondary chain, it can no longer ensure that the coins in tx cannot be double spent in the time it takes to the PoS to confirm. However, if double spending occurs (which means that the PoR blockchain has forked), p_j can claim (as part of the recovery mechanism) the original amount v through the collateral.

At first sight the above might appear as a special, Layer 1, version of a payment channel [pay]. However, it is not hard to verify that unlike payment channels, the above allows p_i to make instant payments to p_j several times with a single pledge, as long as the interval between any two such payments is long enough for the PoS chain to have the time to detect forks.

Notwithstanding, the novelty in the above mechanism becomes more evident once the above pledge is replaced by a mechanism that allocates system funds (called a *treasury*) as collateral, with the amount allocated to each party dependent on their reputation. Intuitively, we use the parties’ reputation analogous to how financial services use the concept of credit score. Specifically, the system can pledge for each party an amount (depending on the reputations) that acts as the above pledged coins. The party can then pledge (without needing to deposit their own funds as collateral) to a recipient, or several recipients if they specify the associated fractions of the pledged amount for each. By properly tuning the pledged amount for each party using the reputations, we can limit the total risk of the system (i.e., the

amount that could be lost if all parties misbehaved); furthermore, we can prove that the strategy of any (malicious) node to double-spend is dominated by the node playing honestly. We give the relevant details and proofs in Section 7. We believe that the above demonstrates how our tiered PoR-with-PoS-fallback paradigm can enable DeFi mechanisms, beyond what is currently achieved by monolithic (single assumptions) or Layer-2 solutions. Devising additional such mechanisms, as well as additional applications of monetized reputation is an interesting research direction. We refer to Tab. 1 for a schematic summary of our contribution.

3 Related Work

The most relevant work is [KOZ20], from which we extend the design of our protocol. We achieve several improvements: First, in [KOZ20], when a safety failure in PoR is detected, all execution falls back to the PoS; in contrast, we propose a recovery mechanism to restart the PoR while removing misbehaving part(ies). This is akin to the idea of *accountable safety* (e.g., [GCN⁺25,NTT22,LPR25,NTT24]), where a fraction of corrupt parties can be identified and removed [LPR25,GCN⁺25] when a safety violation happens (note, due to the nature of the PoR, it is only possible to cause a fork in the event of dishonest majority with at least one identifiable, double-signing party—thus we can support removing this party). Our two-chain setting offers us additional benefits, however. In addition to identifying and removing corrupt parties, our protocol can mitigate all safety violations—even in the event of a safety violation in the PoR chain, payments using our ‘safe transactions’ will never be reverted after recovery.

We remark that accountable (transaction) liveness is known to be harder to achieve than accountable safety described above [LPNRZ25]. In fact, accountable liveness (in the single-blockchain setting) is shown to be impossible in the case of dishonest majority (which we must consider when the PoR chain assumption is broken). Nevertheless, more than detecting safety failures, our work considers also the case when specific transactions may be censored on PoR, by taking inspiration from the two-tiered approach of Thunderella [PS18] (more comparisons below). In contrast to [KOZ20], our protocol uses in a (nearly) black-box manner the underlying PoR protocol. Lastly, we extend the security analysis of [KOZ20] by proposing an incentive structure, such as block rewards, and prove that corrupt parties have no incentive to misbehave.

We compare below with other relevant works. Recently, Trustboost [SWK⁺23] presented a protocol in the validator/client setting that reaches consensus by using in a black-box (only read/write) manner some m blockchains, of which f number of them can be faulty. Notably, they proved that under assumption of partial synchrony and their black-box model, it is impossible to reach consensus if $m \leq 3f$. Intuitively, this follows from the analogous impossibility in consensus literature. In this work, we circumvent this impossibility, by achieving a weaker *payment finality* property with $m = 2, f = 1$ —this property informally says that once a ‘safe’ transaction appears in a block held by an honest party, the transaction’s recipient will eventually receive the same amount of coins described by the transaction’s output (even if safety may be violated). We do so by modifying the following: our protocol, while also using the underlying PoR and PoS blockchains in a somewhat black-box way: we do not change how their block creators are chosen or how they reach consensus about a block, but we parametrize them by defining e.g., a transaction validity predicate, as well as dictating how stake and reputation are computed. Moreover, we consider synchronous network. Interestingly however, our protocol also achieves a *stronger* property than previous work: if the safety of our protocol’s underlying PoR blockchain fails, we not only detect such failure, but can also restart the PoR chain and remove at least one dishonest party. This means that we could recover from any inaccuracies in our reputation system (the assumption underlying the PoR blockchain’s safety).

Other previous works also construct blockchains combining (several) underlying blockchain protocols. Thunderella [PS18] achieves instantaneous confirmation in the optimistic case of 3/4 honest majority of an agreed-upon committee and safety when its underlying (PoW) blockchain has honest majority. Intuitively, Thunderella combines chains—one slow and one fast. The fast path requires signatures from 3/4 parties from an agreed-upon committee. In parallel, a slow (e.g., PoW) chain is run, on which parties post hashes of fast blocks or transactions which they would like to see on the fast chain. These allow parties to detect failures in the fast chain and fallback to the slow one. Our liveness property uses a similar method as Thunderella, but we guarantee safety in case either of the fast (PoR) or slow (PoS) chains may fail. This is in contrast to Thunderella, where (even) a temporary fork on any of its two

chains can result in a safety violation. In addition, we also outline in Sec. 7 how to incentivize parties (i.e., reward them) to follow our protocol. Meshcash [BHMN21] devised a race-free mechanism, termed tortoise and hare consensus to take advantage of a DAG-based, fast asynchronous but possible buggy protocol to improve liveness and scalability, while relying on a slower synchronous consensus mechanism to ensure safety. The work of Tas et al. [THTY23] considers the same problem as ours, but is somewhat incomparable. They treat safety and liveness as properties that can be separately broken, whereas we consider the case where the underlying assumption of the PoR (or PoS) can be broken (in which case, *both* safety and liveness may not hold). In particular, [THTY23] achieves safety if at least one underlying blockchain is safe, and liveness if all blockchains are live.

Works that achieve better safety by checkpointing blocks (e.g., on another chain) [KK21,RKK⁺22,SWKV21,TTG⁺23] are similar to how we detect safety failures in our PoR chain; however, in our work, we allow the checkpointing mechanism (i.e., our PoS chain) to also fail to satisfy safety. Moreover, even in the case where our fast chain (PoR) fails, we still achieve (instant) finality for *transaction payments*, though with worse liveness. Compared to payment channels [pay], which allow for off-chain payments by first ‘locking’ some amount of coins on chain, this work achieves payment finality by enabling the sender to pledge coins to the recipient. In contrast to the locked coins in payment channels, this amount of pledged coins (which we call *collateral*) is not used up until a transaction sender double-spends—which does not happen when the sender is honest (hence, pledge coins can be re-used for multiple payments whose total amount is more than the pledge coins). Finally, we note that a mechanism for guaranteeing quick payments between a merchant and a customer is also proposed in [LJ23]. In this, a third party, upon receiving a transaction (that is not yet settled on the chain) from the customer directed to a merchant, makes the quick payment to the merchant over the payment channel, effectively taking the risk that the customer transaction may not settle on-chain (i.e., the third party works as a credit card company in some sense) [LJ23]. In our approach, there is no third party, and there is no risk of any party losing money for transactions issued using our *safe transactions* mechanism.

4 Preliminaries and Model

We use “=” to denote equality of two different elements (i.e. $a = b$ then...) and “ $\leftarrow$ ” as the assignment operator (e.g. to assign to a the value of b we write $a \leftarrow b$). A randomized assignment is denoted with $a \xleftarrow{\$} A$, where A is a randomized algorithm and the randomness used by A is not explicit. We model the hash function as a random oracle. Following [GKL15,DGKR18], we prove our result in the q -bounded model, in which all our theorems are proven assuming that the adversary does not exceed a fixed (polynomial) number of queries to the random-oracle.

Round, Slot, Epoch. Synchrony Assumption. In the blockchain protocol, we consider time as divided into discrete units called **rounds**. For simplicity we assume the parties are synchronized—that is, honest parties agree on which round it is. We consider blockchain protocols which run in a series of **slots**, which are contiguous lengths of rounds (may be of fixed or dynamic length¹⁰) in which a lottery is run to determine the parties who would be responsible for creating the block for that slot. An **epoch** instead is a fixed length of contiguous slots. For example, if the epoch length of a protocol is R , then epoch 1 consists of slots $\{1, \dots, R\}$, epoch 2 consists of slots $\{R + 1, \dots, 2R\}$, and so on. In the protocols we consider, parties communicate over a network of authenticated multicast channels.

4.1 Abstract Blockchain Protocol

At a high level, a blockchain system is a cryptographic protocol executed by multiple parties, where the purpose of the parties is to agree on a set of inputs, called *transactions* and include this transaction in a *block*. This block is added to previously added blocks, starting from a special genesis block, that we denote with $\mathcal{G}$.

The main properties that a blockchain protocol guarantees are that, as long an underlying assumption holds (i.e., the majority of the computational power is controlled by the honest parties), then transactions

¹⁰ For example, Algorand [GHM⁺17] is currently implemented with slots that vary in length of time, dependent on network conditions.

generated by honest parties eventually become part of the chain (*liveness*), and that all honest parties always eventually agree (i.e. have the same view) on the content of the chain (*safety*). In this work, we will use a simple abstraction of a blockchain protocol. This is described in Fig. 4.3. At a high level, the blockchain protocol is parametrized by a few procedures that we discuss here.

- **GetChain**, which allows each party to get access to their local view of the chain.
- **TxSubmit** is a multi-party protocol used by users to submit a transaction tx (e.g., this is the procedure that diffuses the transaction over the network)
- **Pstatus** is a procedure used by each party to obtain the most recent system status. For example, in a PoS chain, the status is represented by the stake distribution of the parties, or in the PoR chain, the status represents the reputation vector. **Pstatus** requires each party to input the view of the chain in every epoch and the current epoch number t . How **Pstatus** works depends on the type of blockchain we are using.
- **BlockCreate**. This is an interactive multi-party protocol that allows the creation of a new block that will be added to the chain. **BlockCreate** is run by each party at the beginning of every new slot using as input a seed (this is the seed used for the lottery to decide who are the block leaders are, more on this later). **BlockCreate** takes as input the output obtained by **Pstatus**, the local chain of the caller, the slot index, and a set of transactions, and it returns the new block, a set denoting the block leaders, and a certificate. The certificate contains the signatures of the block hash computed by the block leaders.
- **certify** is used to check if two hash headers are valid headers of two consecutive blocks in the chain.
- **checklead** is a procedure that allows verifying if a party was elected as a leader (often referred to as block creator) to generate a specific block.
- **validity** takes a blockchain view **chain**, a transaction tx , a buffer **buffer**, and some auxiliary information **aux**, and determines whether a transaction is valid or not concerning the transactions in $\text{chain} \cup \text{buffer}$. The exact specification of this validity predicate depends on the application for which the blockchain is used.
- **isvalidchain** is the predicate that checks whether a **chain** is a valid hash chain, i.e., the chain validation predicate in [GKL15], and that every transaction tx in **chain** is valid (the validity of the chain can be tested using the validity predicate above setting **buffer** = $\emptyset$.)
- **GOracle** is an oracle that when called, returns the (same) genesis block (i.e., the first block of the chain) to every party invoking it. **GOracle** is parametrized by an initial set of transactions and by an initial seed. In this work, we follow [CKKZ20] and consider a relaxed version of the genesis block oracle. We indeed allow the adversary to bias the seed, and get an advantage over the honest parties, by seeing the seed before the honest parties do. How much the seed can be biased, and how much time in advance the adversary can see the seed compared to the honest parties, is defined by the parameters m and **delay** respectively. For example, when **delay** = 0 and $m = 1$ correspond to the case where there is only one candidate genesis block and all the parties can see it at the same round. We refer to Fig. 4.2 for the formal description of the oracle.
- **Pseed** is an oracle that returns the seed that will be used to run the lottery (e.g., the hash of previously generated blocks).

Figure 4.1: The seed generation algorithm $\text{Pseed}^{\text{PoR}}$

Let t be the current slot, and let **chain** be the local chain of the party in the slot $t - 1$. Let B be the block added to **chain**. Return $H(B)$.

Figure 4.2: The genesis functionality **GOracle**

Parameters. The functionality is parametrized by **delay** the genesis block B^{gen} initialized with a default value $\perp$ and a set of transactions **Txs**, and default starting time τ . The oracle is also parametrized by a flag **corrupted**. If **corrupted** = 0 then the oracle behaves as described below, otherwise, the full control of the oracle is given to the adversary.

Let R denote the slot number in the moment the oracle is invoked.

- If $I = \text{gen_genesis}$ is received from the adversary $\mathcal{A}$ then sample $\text{seed}_i \xleftarrow{\$} \{0, 1\}^\lambda$ and add seed_i to the initially empty set **GB**, and return seed_i to the adversary.
- If $I = \text{get_genesis}$ is received from an honest party $p_i \in \mathcal{P}$ do the following
 - If $B^{\text{gen}} \neq \perp$ then return B^{gen} to p_i .

- If $B^{\text{gen}} = \perp$ and $R - \tau > \text{delay}$ then sample $\text{seed} \leftarrow \{0, 1\}^\lambda$, define the genesis block as $\mathcal{G} = (\text{seed}, \text{TxS})$, set $B^{\text{gen}} \leftarrow \mathcal{G}$ and send B^{gen} to p_i .
- If $I = (\text{set_genesis}, \text{seed}')$ is received from the adversary do the following
 - If $(R - \tau) \leq \text{delay}$ and $\text{seed}' \in \text{GB}$ then set $B^{\text{gen}} := (\text{seed}', \text{TxS})$.
 - Else, return $\perp$ to the adversary.

Figure 4.3: Abstract Blockchain Protocol

Parameters and setup

- Slot length SlotLen denoted in terms of number of rounds.
- Each party p_i running the protocol, has associated an address addr_i (a verification key of a signature scheme).
- At the beginning of the protocol the honest parties invoke GOracle with the command `get\_genesis` thus obtaining a genesis block $\mathcal{G}$.

Each party p_i keeps track of the following:

- t : Current slot (all parties agree on the slot number due to synchronicity)
- **chain**: Current chain obtained via the invocation of `GetChain`.
- b : List of blocks and their creators/leaders, indexed by slot. Initialized to $b[s] = \emptyset$ for all s . We only store block creators/leaders in position s , if in the slot s a block was successfully created.
- **Chains** contains the view of the chain of the party in every slot.
- An initially empty set **buffer**.

In each slot t , each party executing the protocol, on input a set of transaction TxS does the following:

1. Run $\text{Pstatus}(\text{Chains}, t)$ thus obtaining **status**, and invoke Pseed thus obtaining **seed**.
2. For every $\text{tx} \in \text{TxS}$, if $\text{validity}(\text{tx}, \text{chain}, \text{buffer}, \text{aux}) = 1$ then add tx to **buffer**.
3. Run `BlockCreate` on input $(\text{chain}, \text{status}, \text{seed}, t, \text{buffer})$ thus obtaining (B, C, cert) . Let B_n be the most recent block added to **chain**.
4. If for every $\text{addr} \in C$ $\text{checklead}(\text{status}, \text{seed}, \text{addr}, \text{cert}, |\text{chain}| + 1) = 1$ and $\text{isvalidchain}(\text{chain} \| B) = 1$ and $\text{certify}(\text{status}, H(B), H(B_n), \text{seed}, |\text{chain}| + 1, \text{cert}) = 1$ then continue as follows, else reject the block and stop.
5. $\text{chain} \leftarrow \text{chain} \| B$.

On request `GetChain`: Return **chain**

Having defined the syntax of a blockchain protocol, we now state the security properties that we may want from such protocol. Before doing that, we define a predicate ASP . This predicate takes as input a slot number and returns 1 or 0. We require the standard properties that one expects from a blockchain protocol to hold, conditioned on the assumption ASP being equal to 1. In a nutshell ASP tells us whether the blockchain protocol we are running can be trusted due to the underlying assumption being valid. By underlying assumption here, we mean, for example, that there is an honest majority of the nodes running the protocol, or that the majority of the stake is controlled by honest parties. The exact specification of ASP depends on the protocol. The predicate ASP is of course never used by the parties running the protocol, but it is used to state our theorems and definitions as it allows us to specify what properties can be guaranteed depending on whether, in a given time frame, ASP holds or not. This will be particularly relevant when defining the security of our hybrid protocol, where the liveness and safety parameters will depend on which of the two blockchain protocols is safe (i.e., which of the two assumptions is valid). We revisit the notion of safety, block liveness, and transaction liveness to capture this aspect. These definitions are formally described in Appendix B.

Reputation and Stake. For our approach, we will rely on a PoS and a PoR chain, and we will formally describe the properties of our system depending on which of the chains is *safe*. In particular, we will denote the assumption predicate of the PoS chain with ASP^{PoS} and the assumption for the PoR chain with ASP^{PoR} . To define the meaning of this predicate, we recall the definition of reputation system from [KOZ20] (see Appendix B): Informally, a reputation system Rep describes the probability distribution on the honesty/dishonesty of reputation parties, described by a reputation vector $(h_1, \dots, h_m)$. For example, if according to Rep , we have $\Pr[(h_1, \dots, h_m) = (0, \dots, 0, 1)] = 0.6$, then with probability 0.6, everyone except P_m is dishonest. Following [KOZ20], we call a reputation system Rep *feasible* (see

Definition B.6 in Appendix B) if there exists a (PPT) sampling algorithm A which, with overwhelming probability, chooses a subset of parties which has honest majority.

We make a distinction between real-world reputation Rep^{real} (which is the actual, ground-truth probabilities that parties are honest/dishonest) and system-extracted reputation Rep^{sys} . We say that Rep^{sys} is *accurate* if the probability distribution actually describes (within negligible error) the real probability in which parties are honest/dishonest (i.e., Rep^{real}). Conversely, we call a reputation system Rep^{sys} that is not accurate, *inaccurate*. For brevity in our theorems, we introduce the following predicates, ASP^{PoR} and ASP^{PoS} : We say that for a PoR chain, in a slot t $\text{ASP}^{\text{PoR}}(t) = 1$ if and only if the reputation vector Rep is ϵ -feasible. For a PoS stake instead, we say that in a slot t $\text{ASP}^{\text{PoS}}(t) = 1$ if and only if the fraction of honest stake is a constant fraction above $2/3$. We refer the reader to Appendix B.1 for more details on reputation systems.

5 Security properties, and main tools

In this section, we describe our main tools: a PoR and a PoS blockchain system that follow the abstraction proposed in Section 4.1. We argue that our abstractions are realized by the PoR protocol proposed in [KOZ20] and by almost any PoS protocol. Before describing the exact properties we want from the PoR and the PoS chain, we highlight how our protocol will work and what are the main security properties that we guarantee.

At a very high level, in our protocol each party will run the two protocols blockchains in parallel, using the PoR protocol as the primary chain (where the transactions will be posted), and using the PoS protocol *only* to checkpoint the hash of the block generated via the PoR protocol. The PoR chain will be the only chain actually managing the coins and the treasury (more on the treasury later). Hence, any time that a party p_i wants to send money to p_j , p_i signs and issues on the PoR chain a transaction of the following type (`STANDARD, addri, addrj, v`), where `addri` and `addrj` denote the addresses of p_i and p_j respectively, and v is the number of coins that p_i wants to send to p_j . The main property we guarantee in our system is that if the assumption underlying the PoS chain (i.e., ASP^{PoS} holds) is valid then the following happens.

- If ASP^{PoR} (i.e., the assumption that underlines the PoR chain) holds in a sufficiently long time interval $[t_1, \dots, t_2]$, then the transaction will be confirmed at the speed of the PoR chain in that interval (i.e., we get the transaction liveness of the PoR chain).
- If ASP^{PoR} does not hold, then in the worst case transactions will be confirmed at the speed of the PoS chain (plus an additional constant parameter).
- Regardless of whether ASP^{PoR} holds or not, safety is always guaranteed (the parties will eventually agree on a non-trivial common prefix).

Moreover, if ASP^{PoR} holds, we get the safety and liveness properties of the PoR chain, regardless of whether ASP^{PoS} holds.

In a nutshell, we design a blockchain protocol that guarantees security as long as only one of the blockchain protocols is guaranteed to work as prescribed. We stress that the security of the protocol is guaranteed assuming that throughout the lifetime of the system, if an assumption fails, then the other assumption is always guaranteed to hold (i.e., we do not consider cases where the two assumptions alternate their validity).

5.1 Safe transactions

Another interesting property that we prove in our system, is that when the PoR assumption does not hold (hence forks and double spending could be performed), honest parties that accept the special type of transactions, that we call *safe transaction*, are guaranteed to get refunded. These transactions are particularly interesting, as they can be confirmed at the speed of the PoR chain, even if there is an undetected fork in the PoR chain, and still guarantee that the receiver who confirmed the transaction will eventually get refunded.

The refund will be issued via a treasury. Managing this mechanism requires some care as in the event the PoR chain forks, potentially unlimited double-spending attacks could be performed. We will discuss how this mechanism works in the next section (and refer to the introductory section for a glimpse of this aspect of our protocol). We provide a summary of the properties of our protocol in Tab. 1, and we will state formally all the properties satisfied by our protocol in Section 6.

	$0, \dots, t_1$	$t_1, \dots, t_2$	$t_2 + \Delta, \dots, t_3$	Performance/security in the interval $t_2 + \Delta, \dots, t_3$
PoR	Yes or No	No	Yes	PoR chain
PoS	Yes	Yes	Yes	and <i>safe transactions</i>
PoR	Yes	Yes	Yes	PoR chain
PoS	Yes or No	Yes or No	Yes or No	and <i>safe transactions</i>
PoR	Yes or No	No	No	PoS chain
PoS	Yes	Yes	Yes	

Table 1: With *Yes* and *No* we denote if a chain is secure or it is not secure respectively. The column indicates slot intervals. For example, the first row captures the case where the PoR chain is compromised, but then it regains its security. In this case, the performances (in terms of safety and liveness) of our protocol, after some fixed recovery time Δ , come back to the PoR performances. We write *Yes or No*, it means that the performances of the protocol are independent of whether that blockchain is secure or not.

5.2 Main tools

We now provide a detailed description of the main tools used to design our protocol. For the PoR (PoS) abstraction, denoted with Π_{PoR} (Π_{PoS}) we name any predicates, procedures, and parameter x of the abstract protocol, with x^{PoR} (x^{PoS}). From our abstract PoR protocol Π_{PoR} we require safety with safety parameter $\text{co}^{\text{PoR}} = 1$, instant finality, block liveness (with liveness parameter bliv^{PoR}), and transaction liveness (with liveness parameter liv^{PoR}). Similarly, for our abstract PoS protocol Π_{PoS} , we denote the safety parameter with co^{PoS} , the block liveness parameter with bliv^{PoS} , and the transaction liveness parameter by liv^{PoS} . We now just need to define the type of transactions accepted by each blockchain protocol, how the validation predicates work, how the genesis block oracles are initialized, and what the relationship is between the PoS and the PoR parameters. Let the slot size of the PoR and of the PoS protocol be respectively $\text{SlotLen}^{\text{PoR}}$ and $\text{SlotLen}^{\text{PoS}}$. We consider the setting where $\text{SlotLen}^{\text{PoR}} \ll \text{SlotLen}^{\text{PoS}}$ as this captures the setting we are interested in, where the primary chain (the PoR chain) is faster than the secondary chain (PoS). To simplify the description of our protocol we assume that $\text{SlotLen}^{\text{PoS}} = \ell \text{SlotLen}^{\text{PoR}}$ for some parameter $\ell = \text{poly}(\lambda)$ (but this simplification can be easily avoided).

We require both chains to enjoy the following additional property that we call Δ -*External Signaling*. At a high level, this property requires that if an event external to the blockchain occurs, and all honest parties are aware of this event, then the event will be reported on the blockchain. Most existing blockchains satisfy this property, as the nodes running the chain can vote to claim whether a certain event occurred or not, and if the majority of the nodes support the event (majority in terms of stake or reputation), then the event is deemed valid.

Property 5.1 (Δ -External Signaling). Let $\mathcal{O}$ be a deterministic oracle that, when queried by a party p , with a string q (denoting a certain event), returns 1 (denoting that the event has actually happened) or 0 (denoting that the event has not happened). The property of *external signaling* for a ledger $\mathcal{L}$ requires the following:

1. If $\text{ASP}(t) = 1$ for every slot $t \in [0, \dots, m]$ and $\mathcal{O}$ returns 1 to all the honest maintainers of $\mathcal{L}$ querying the oracle with some question query in a given time slot $\tau \in [0, \dots, m - \Delta]$, then the transaction $(\text{ORACLE}, \text{query})$ is included a part of the state of $\mathcal{L}$ within Δ time slots (i.e., accepted as a valid transaction).
2. If $\text{ASP}(t) = 1$ for every slot $t \in [0, \dots, m]$ and a transaction $(\text{ORACLE}, \text{query})$ appears in the view of all the honest maintainers of $\mathcal{L}$, then it must be that at least one honest party has invoked $\mathcal{O}$ with input query and obtained 1 as answer in a slot belonging to the interval $t \in [0, \dots, m]$.

We now describe how each individual protocol is parametrized, and then provide a high-level and formal description of our protocol.

5.3 The properties of Π_{PoR}

The genesis functionality $\text{GOracle}^{\text{PoR}}$ is parametrized by $\text{delay} = 2c\text{q}^{\text{PoS}}\ell$. $\text{GOracle}^{\text{PoR}}$ will contain an initial seed, the initial reputation vector, and a set of transactions that contains the initial balances (stake) of the parties in the system.

Other than the properties we have just mentioned, we require the PoR protocol to enjoy this additional property. If $\text{certify}^{\text{PoR}}$ certifies that a hash (h_j) in the chain is subsequent to another hash h_{j-1} , then this implies that in the view of the honest parties running the PoR chain, it is indeed the case that B_{j-1} comes right before B_j with $h_j = H(B_j)$ and $h_{j-1} = H(B_{j-1})$. This property is required to hold only if the assumption underlying the PoR chain holds. More formally:

Property 5.2 (Certificate unforgeability). If $\text{ASP}^{\text{PoR}}(t) = 1$ for every slot $t \in [0, \dots, m]$, then for every j with $0 \leq j \leq m$, no *ppt* adversary can create $B^0, \text{cert}^0, B^1, \text{cert}^1$ with $B^0 \neq B^1$, such that $\text{certify}(\text{status}, h', H(B^0), j, \text{cert}^0) = 1$ and $\text{certify}(\text{status}, h', H(B^1), j, \text{cert}^1) = 1$ where status is obtained from an honest party executing Pstatus for the j -th slot.

Following the protocol proposed in [KOZ20] we also assume that the seed for the lottery is computed by hashing the blocks generated in the previous slot (i.e., Pseed is realized as shown in Fig. 4.1). In some of our theorems, we will also rely on the property of *seed recoverable security*. This property guarantees that the security offered by Pseed (e.g., in terms of unbiasedness) gets restored when the assumption underlying the protocol holds again. We stress that no other requirement is put on any other aspect of the protocol. Indeed, liveness and consistency may be completely unrecoverable even if the assumption that underlines the chain does not hold just for a brief period of time (and it is later restored) [BGK⁺24]. We will make it explicit in the theorem when this property is needed.

We describe the type of transactions that can be parsed by the validity predicate $\text{validity}^{\text{PoR}}$, and provide a high-level description of how $\text{validity}^{\text{PoR}}$ works¹¹. See Fig. 5.1 for a formal description of $\text{validity}^{\text{PoR}}$.

Figure 5.1: $\text{validity}^{\text{PoR}}$ predicate for the PoR protocol Π_{PoR}

$\text{validity}^{\text{PoR}}(\text{tx}, \text{chain}, \text{buffer}, \text{aux})$

Set $n = |\text{chain}|$, and let history be the set of transaction contained in chain .

- If $\text{tx} = (\text{STANDARD}, v, \text{addr}_i, \text{addrs}, \text{id}_i, \sigma_i)$ then
 - Initialize $\text{balance} \leftarrow 0$.
 - For each tx^* in $\text{history} \cup \text{buffer}$ with tx^* do the following
 - If $\text{tx}^* = (\text{STANDARD}, v^*, \text{addr}_i^*, \text{addrs}^*, \text{id}_i^*, \sigma_i^*)$
 - If $\text{addr}^* = \text{addr}_i$, then compute $\text{balance} \leftarrow -\sum_k v^*[k]$.
 - For each k such that $\text{addrs}^*[k] = \text{addr}_i$, compute $\text{balance} \leftarrow \text{balance} + v^*[k]$.
 - If $\text{tx} = (\text{ORACLE}, \text{REWARD}, C, m, \text{cert})$ and $\text{addr}_i \in C$ then $\text{balance} \leftarrow \text{balance} + \text{rew}$
 - If $\sum_k v[k] \leq \text{balance}$, then return 1. Otherwise, return 0.
- If $\text{tx} = (\text{PLEDGE}, \text{addr}_i, \text{addr}_j, \text{id}_i, T, \sigma_i)$ and there is no other transaction with $(\text{PLEDGE}, \text{addr}_i, \cdot, T', \cdot) \in \text{history} \cup \text{buffer}$ with $T' \geq t^{\text{PoS}} + \Delta$ then return 1.

- $\text{tx} = (\text{STANDARD}, v, \text{addr}_i, \text{addrs}, \text{id}, \sigma)$. v is a vector of values, addr_i denotes the address of the sender and addrs is a list of receiver. The transaction specifies that $\text{addrs}[k]$ should receive $v[k]$ coins. σ just represents the signature of the transaction with respect to the public-key addr_i .¹² To validate this transaction, $\text{validity}^{\text{PoR}}$ simply checks if the balance of addr_i is sufficient to pay the receivers by looking at all the accepted transactions so far, and computing the balance of addr_i accordingly.
- $\text{tx} = (\text{PLEDGE}, \text{addr}_i, \text{addr}_j, \text{id}_i, T, \sigma_i)$. This transaction does not specify any amount, but just a sender/receiver address, and an expiration time T . Such a transaction is valid, only if addr_i has not issued any other *pledge* transaction that has still a valid timer. A timer T is valid if $T \geq t + \Delta$, where t denotes the current slot, and Δ is a parameter that we will specify later. This transaction represents a promise that the sender will pay the receiver, even in the event of a fork. In particular, if a receiver has been the victim of a double spending attack due to a fork on the PoR chain, this party can later complain on the PoS chain, and ask for a refund. This refund will be taken from the system treasury, and the refundable amount depends on the reputation of the party with address addr_i (the higher the reputation is, the higher the amount of money that a party can pledge, and hence, be used as a refund). This transaction is what will help us to create what we have called a *safe transaction*. We will provide more details on these mechanisms later.

¹¹ All the transaction are signed, and the validity predicates accepts only transaction that are properly signed. We assume that the check of the signatures is implicit, and avoid restating it any time that the validation of a transaction is needed. Similarly, we assume that each transaction has a unique identifier, and that transactions with the same identifier, coming from the same address, are rejected.

¹² For simplicity we just assume that an address corresponds exactly to a public-key of a signature scheme.

- (REWARD, C, m, cert), this transaction contains a set C of block leaders who generated the m -th block on the secondary chain (the PoS chain). This transaction is validated only if the m -th block of the secondary chain contains a valid checkpoint and the leaders (PoS block creators) are all part of the set C . Looking ahead, this transaction is used to reward the parties that generate a block in the secondary chain. I.e., the existence of this transaction on the PoR chain, means that all the addresses in C are entitled to receive a reward. For simplicity, we assume that this reward is fixed and defined by the parameter **rew**. We will provide more detail on this aspect on Section 6
- $\text{tx} = (\text{REPUTATION-UPDATE}, \text{status}, \text{epoch})$. This transaction is generated by the system and denotes the reputation (**status**) at the epoch **epoch**. In practice, the blockchain is used by the parties to agree on a new reputation vector. If the agreement is successful, then the transaction (REPUTATION-UPDATE, **status**, **epoch**) appears on the chain to denote such an event.

5.4 The properties of Π_{PoS}

The genesis functionality $\text{GOracle}^{\text{PoS}}$ is parametrized by **delay** = 1. $\text{GOracle}^{\text{PoS}}$ contains an initial seed and only one transaction (CHECK-POINT, $H(\mathcal{G}^{\text{PoR}}), 0, \perp$), where $h_0 \leftarrow H(\mathcal{G}^{\text{PoR}})$ and $\mathcal{G}^{\text{PoR}}$ is the genesis block of the PoR chain. We will elaborate more on each transaction considered in our protocol but in a nutshell, the genesis block of the PoS chain contains a checkpoint of the PoR chain. Initially, this checkpoint simply represents the hash of the genesis block of the PoR chain. We do not specify how $\text{BlockCreate}^{\text{PoR}}$ works, except the following. Let (B, C, cert) be the output of $\text{BlockCreate}^{\text{PoR}}$, we require that $\forall \text{addr} \in C \exists \sigma \in \text{cert}$ such that $\text{Ver}(\text{addr}, h(B), \sigma) = 1$ (i.e., we require that the block leaders provide a valid signature of the hash of the created block).

We also require the following property to hold, which guarantees that honest block leaders are selected relatively often.

Definition 5.1 (Chain Quality). *A blockchain protocol that follows the abstraction of Fig. 4.3 satisfies Chain Quality with parameter **cq** if for every $t \in [0, \dots, m]$ with $\text{ASP}^{\text{PoS}}(t) = 1$ and $\text{GOracle}^{\text{PoS}}.\text{corrupted} = 0$ the following occurs: at least every **cq** slot in the interval $[0, \dots, m]$ the output of $\text{BlockCreate}^{\text{PoS}}$, denoted with (B, C, cert) , is such that C contains the address of at least one honest party.*

We also require a property held by most of the existing blockchains, which requires the hash of a block that will be generated relatively in the future to be unpredictable (a similar property is formalized in [ACKZ20]).

Definition 5.2 (Unpredictability). *A blockchain protocol that follows the abstraction of Fig. 4.3 satisfies unpredictability with parameter **cq** if for every $t \in [0, \dots, m]$ with $\text{ASP}^{\text{PoS}}(t) = 1$, no ppt adversary can, at slot t , guess with more than negligible probability the hash of a block that will be generated in the slot $t + \text{cq}$ (with $t + \text{cq} < m$), where **cq** represents the chain quality parameter of Definition 5.1.*

We define formally the validity predicate $\text{validity}^{\text{PoS}}$ in Fig. 5.2, and below we describe the type of transactions that can be parsed by such a validity predicate.

Figure 5.2: $\text{validity}^{\text{PoS}}$ predicate for the PoS protocol Π_{PoS}

$\text{validity}^{\text{PoS}}(\text{tx}, \text{chain}, \text{buffer}, \text{aux})$

Set $n = |\text{chain}|$, and let **history** be then set of transactions contained in **chain**. Parse **aux** as $(\text{recovery}, \text{chain}^{\text{PoR}}, t^{\text{PoR}})$.

If **recovery** = 0 then do the following.

- If $\text{tx} = (\text{CHECK-POINT}, h, j, \text{cert})$. Invoke $\text{FindReputation}(j)$ thus obtaining status^j . If there exists $\text{tx}' \in \text{history} \cup \text{buffer}$ where $\text{tx}' = ((\text{CHECK-POINT}, h', j - 1, \text{cert}'))$ such that $\text{certify}(\text{status}^j, h', h, j, \text{cert}) = 1$ OR $(h = H(\text{chain}^{\text{PoR}}[0]) \text{ and } j = 0)$ then return 1, else return 0
- If $\text{tx} = (\text{COMPLAINT}, h^*, \text{cert}^*, j)$ then do the following
 - If there exists $\text{tx}_{j-1}, \text{tx}_j \in \text{history}$ where $\text{tx}_{j-1} = (\text{CHECK-POINT}, (h_{j-1}, j - 1, \text{cert}_{j-1}))$ and $\text{tx}_j = (\text{CHECK-POINT}, (h_j, j, \text{cert}_j))$ then continue as follows, else return 0.
 - If $\text{certify}(\text{status}^j, h_{j-1}, h^*, \text{cert}_j^*) = 1$ and $h^* \neq h_j$, with $\text{status}^j = \text{FindReputation}(j)$ then return 1.
- If $\text{tx} = (\text{BLOCK-REQUEST}, i)$ and $\exists (\text{LIVENESS-VIOLATION}) \in \text{history}$ then return 1, else return 0.
- If $\text{tx} = (\text{BLOCK-REQUEST-RESPONSE}, i, B)$ and $\exists (\text{BLOCK-REQUEST}, i) \in \text{history}$ then return 1, else return 0.

Return 0. If $\text{recovery} = 1$ then do the following

- If $\text{tx} = (\text{PAYBACK}, \text{addr}_j, \text{tx}', \sigma')$ and $\text{Ver}(\text{addr}_j, \text{tx}', \sigma') = 1$ and $\text{tx} \notin \text{history} \cup \text{buffer}$ and $(\text{PAYBACK}, \cdot, \text{tx}', \cdot) \notin \text{history} \cup \text{buffer}$ then continue as follows, else return 0.
 If $\text{tx}' = (\text{STANDARD}, v, \text{addr}_i, \text{addrs}, \text{id}_i, \sigma)$ then define $m := (v, \text{addr}_i, \text{addrs}, \text{id}_i)$.
 If $\text{Ver}(\text{addr}_i, m, \sigma_i) = 1$ and $\exists k$ s.t., $\text{addr}_j = \text{addrs}[k]$ then continue, else return 0.
 Set $n^{\text{PoR}} = |\text{chain}^{\text{PoR}}|$, and let $\text{history}^{\text{PoR}}$ be the set of transactions contained in $\text{chain}^{\text{PoR}}$.
 If $\exists (\text{PLEDGE}, \text{addr}_i, \text{addr}_j, \text{id}_i^*, \sigma_i^*, T^*) \in \text{history}^{\text{PoR}}$ with $T^* \geq t^{\text{PoR}}$ and $\text{TreasuryAllocation}(\text{addr}_i, \text{status}^{\text{PoR}}, \text{chain}^{\text{PoR}}) \geq v[k]$ then return 1
- If $(\text{STANDARD}, v, \text{addr}_i, \text{addrs}, \text{id}_i, \sigma_i)$ then continue as follows.
 Let history be the sequence of all the transactions of the form $(\text{PAYBACK}, \cdot)$ in chain . Let addr_i, S_i denote the balance of party p_i computed using the transactions $(\text{STANDARD}, \cdot)$ in $\text{chain} \cup \text{chain}^{\text{PoR}} \cup PB$ where PB contains all the transactions tx' such that $\exists (\text{PAYBACK}, \cdot, \text{tx}', \cdot) \in \text{history}$.
 Compute $\text{spent}_i \leftarrow \sum_k v[k]$ and if $\text{spent}_i \leq S_i$ then return 1, else return 0.

Return 0.

FindReputation(i)

Let $\text{tx} = (\text{REPUTATION-UPDATE}, \text{status}, \text{epoch})$ be the latest transaction of the form $(\text{REPUTATION-UPDATE}, \cdot)$ added to chain at least Δ slots ago such that $i \in \text{epoch}$.^a If such a transaction exists, then return status , else return $\perp$. and return status .

^a We recall that we assume that every cert includes a transaction of the form $(\text{REPUTATION-UPDATE}, \text{status}, \text{epoch})$, denoting the reputation was used to compute the block associated to that certificate.

- $\text{tx} = (\text{CHECK-POINT}, h, j, \text{cert})$. This transaction is used to checkpoint block-hashes of the PoR (the primary) chain on the PoS (the secondary) chain. In particular, the transaction contains the hash h of a block generated in the PoR chain (recall that this transaction is for the PoS chain), j is the block index, and cert represents the certificate that the block is actually the j -th block of the PoR chain. This transaction will be accepted, only if $\text{certify}^{\text{PoR}}$ returns one (recall that $\text{certify}^{\text{PoR}}$ is guaranteed to return one if a hash does indeed correspond to a valid PoR block). In the description of the protocol, we assume for simplicity that cert also contains the reputation vector status used to generate the j -th block. This allows us to export the reputation of the PoR on the secondary chain¹³.
- $\text{tx} = (\text{COMPLAINT}, h^*, \text{cert}^*, j)$. This transaction is used when a party sees a check-point with hash value $h \neq h^*$, with respect to the same index block (i.e., a party noticed that his local view of the secondary chain is different from the view of the parties who did the checkpointing). If this transaction is accepted, then all the parties running the PoS chain will understand that a fork happened in the PoR chain, and will go in a so-called *recovery mode*. We stress that the only way that a complaint can be accepted is if indeed there is a fork on the primary chain (hence ASP^{PoR} does not hold anymore). Indeed, when ASP^{PoR} holds, then no valid complaint can be generated due to the property of *certificate unforgeability* (Property 5.2). We will elaborate more on this in the next section.
- $\text{tx} = (\text{PAYBACK}, \text{addr}_j, \text{tx}, \sigma)$ this transaction is issued by an honest party p_j only in the following situation
 - p_j is in *recovery mode*.
 - p_j accepted a transaction $\text{tx}' = (\text{STANDARD}, \text{addr}_i, \cdot)$ that was discovered to be part of a fork.
 - let t be the slot at which p_j accepted the transaction tx , then the PoR chain contained a transaction $\text{tx} = (\text{PLEDGE}, \text{addr}_i, \text{addr}_j, \text{id}_i, T, \sigma_i)$ with $T \geq t + \Delta$.

In summary transaction can be accepted only when in *recovery mode*. If the transaction is accepted, it will allow p_j to obtain whatever amount was specified in tx' , by either taking the coins from addr_i account or from the treasury system if this money is not available (mode detail will follow). This is another transaction that we will use to support *safe transactions*.

- $\text{tx} = (\text{LIVENESS-VIOLATION})$. This transaction appears on the ledger only when the majority of the parties have noticed a liveness violation on the primary (PoR) chain. In practice, this is realized by making parties vote about whether they think the PoR chain has a liveness issue. As mentioned, we abstract this voting aspect by relying on Property 5.1
- $\text{tx} = (\text{BLOCK-REQUEST}, i)$. This transaction is accepted only if the ledger has already accepted a transaction $\text{tx}' = (\text{LIVENESS-VIOLATION})$, otherwise it is ignored. The transaction is issued by a party

¹³ Note that it is not necessary that every certificate contains the reputation vector. The reputation vector can be included in cert only when there is an update of the reputation.

that did not see the i -th PoR block (this can happen only when the security of the primary chain has been violated), and requests this block to appear on the secondary chain (PoS). An honest party that sees such a transaction, which instead has seen the i -th PoR block B , can react by issuing the transaction `BLOCK-REQUEST-RESPONSE`, i, B).

Instantiations of Π_{PoR} and Π_{PoS} below. We instantiate Π_{PoR} with the PoR protocol of [KOZ20] (without its fallback mechanism to a PoS chain). Intuitively, Thm. 5.2 holds since when the assumption underlying the PoR security holds (i.e., accuracy of the reputation system), honest PoR parties agree on the latest block without ambiguity. This is guaranteed by the fact that a public lottery is run to select n committee members, and a block is accepted if the majority of the committee members sign the block. An honest member of the committee will never sign two conflicting blocks for the same slot. Hence, a certificate in this case can be thought as been the signature of all the committee members.

Any PoS protocol where chain quality, chain growth, and safety hold (under assumption on the honest (super-)majority of stake and synchrony), will suffice as our fallback chain Π_{PoS} . This includes, for example, the Algorand [GHM⁺17] or Cardano/Ouroboros [KRDO17] PoS protocols. Note that the choice of PoS can have different advantages and disadvantages—for example, Algorand satisfies *instant finality* (all transactions that appear on a valid block are final) [alg], but is designed assuming a higher fraction of honest parties.

6 Our Hybrid PoR/PoS protocol

To describe our protocol, which we denote with Π , we use the PoR and PoS abstract protocols that we have introduced in Section 5.3 and Section 5.4, which we have denoted with Π_{PoR} and Π_{PoS} respectively.

Figure 6.1: Main Protocol Π

Setup – All the parties have access to a genesis block of the PoR and PoS chain, these are respectively denoted with $\mathcal{G}^{\text{PoR}}$ and $\mathcal{G}^{\text{PoS}}$. The $\mathcal{G}^{\text{PoS}}$ contains the transactions $\text{tx}_0 = (\text{CHECK-POINT}, H(\mathcal{G}^{\text{PoR}}), 0, \perp)$, where $h_0 \leftarrow H(\mathcal{G}^{\text{PoR}})$. – Every party has the parameter $\Delta \leftarrow 2\text{liv}^{\text{PoS}}$, and it maintains initially empty sets $\text{buffer}^{\text{PoS}}$ and $\text{buffer}^{\text{PoR}}$ and an initially empty list chain^Π (this denotes the local chain of a party). All the parties also share two distinct values dspos, and dspor, that are used to separate the domains of the queries to the hash function (i.e., dspos for the queries related to the PoS chain, and dspor for the queries related to the PoR chain). Each party has a flag flag^{liv} initialized to 0, which represents whether the party has detected a liveness violation in the PoR chain (i.e., whether a valid PoR block has been received at the expected time.) – rectime denotes the duration of the recovery time in PoS slots. This can be any value such that $\text{rectime} \geq 5\text{liv} + 2\text{cq}^{\text{PoS}}$. – The protocol runs in a slot. In particular, we will distinguish between PoR and PoS slots. For simplicity, we assume that one PoS slot corresponds to ℓ PoR slots, for some parameter ℓ <i>Normal mode: protocol maintainers</i> In every PoR slot denoted with t^{PoR}, each party, on input a set of transactions TxS^{PoR} does the following. 1. For every $\text{tx} \in \text{TxS}^{\text{PoR}}$, if $\text{validity}^{\text{PoR}}(\text{tx}, \text{chain}^{\text{PoR}}, \text{buffer}^{\text{PoR}}, \text{aux}) = 1$ then add tx to $\text{buffer}^{\text{PoR}}$. 2. Let B^{PoR^*} be the most recent block added to chain during the slot interval $[0, \dots, t^{\text{PoR}} - 1]$, compute $\text{seed} \leftarrow H(B^{\text{PoR}^*} \parallel \text{dspor})$. 3. Invoke $\text{Pstatus}^{\text{PoR}}(t^{\text{PoR}})$ thus obtaining $\text{status}^{\text{PoR}}$. 4. Run $\text{BlockCreate}^{\text{PoR}}$ on input $(\text{chain}^{\text{PoR}}, \text{status}^{\text{PoR}}, \text{seed}^{\text{PoR}}, t^{\text{PoR}}, \text{buffer}^{\text{PoR}})$ thus obtaining $(B^{\text{PoR}}, C^{\text{PoR}}, \text{cert}^{\text{PoR}})$. 5. Let B_n^{PoR} be the most recent block added to $\text{chain}^{\text{PoR}}$. 6. If for every $\text{addr} \in C^{\text{PoR}}$ $\text{checklead}^{\text{PoR}}(\text{status}^{\text{PoR}}, \text{seed}^{\text{PoR}}, \text{addr}, \text{chain}^{\text{PoR}} + 1) = 1$ and $\text{isvalidchain}^{\text{PoR}}(\text{chain}^{\text{PoR}} \parallel B^{\text{PoR}}) = 1$ and $\text{certify}^{\text{PoR}}(\text{status}^{\text{PoR}}, H(B^{\text{PoR}}), H(B_n^{\text{PoR}}), \text{seed}, \text{chain}^{\text{PoR}} + 1, \text{cert}^{\text{PoR}}) = 1$ then – Set set $b^{\text{PoR}}[t^{\text{PoR}}] \leftarrow (B^{\text{PoR}}, C^{\text{PoR}})$ – Create the transaction $\text{tx} = (\text{CHECK-POINT}, H(B^{\text{PoR}}), \text{chain} , \text{cert}^{\text{PoR}})$ and invoke $\text{TxSubmit}^{\text{PoS}}(\text{tx})$. Else (i.e., a valid block has not been generated) do the following – Set $\text{flag}^{\text{liv}} = 1$ and invoke the oracle $\mathcal{O}^{\text{liv}}$ of Π_{PoS} (we refer to Fig. 6.2 for the description of the oracle) with the transaction <code>LIVENESS-VIOLATION</code>.
--

- If a message (or multiple messages) of the type (NEW-BLOCK, i, B^*) is received from the network, with $i \geq t^{\text{PoR}} - 2$ then set $b^{\text{PoR}}[i] \leftarrow B^*$.
 - For each $i \leq t^{\text{PoR}} - 2$ such that $b^{\text{PoR}}[i] = \perp$ (i.e., no valid block has been generated or received from any other party in relation to the slot i) then check if on the PoS ledger a transaction BLOCK-REQUEST-RESPONSE, i, B) has been posted. If this is the case, then set $b^{\text{PoR}}[i] \leftarrow (B, \cdot)$. If this is not the case, then create the transaction $\text{tx} = (\text{BLOCK-REQUEST}, i)$ and invoke $\text{TxSubmit}^{\text{PoS}}(\text{tx})$.
 - 7. If one (or more) element $(B^{\text{PoR}}, C^{\text{PoR}})$ has been added to $b^{\text{PoR}}[\cdot]$, append this to $\text{chain}^{\text{PoR}}$. Set $\text{aux}[t^{\text{PoR}}] \leftarrow (\text{status}^{\text{PoR}}, \text{seed}^{\text{PoR}})$, $\text{Chains}[t^{\text{PoR}}] \leftarrow \text{chain}^{\text{PoR}}$.
- In every PoS slot t^{PoS} , each party, on input a set of transactions Txs^{PoS} does the following.
1. For every $\text{tx} \in \text{Txs}^{\text{PoS}}$, if $\text{validity}^{\text{PoS}}(\text{tx}, \text{chain}^{\text{PoS}}, \text{buffer}^{\text{PoS}}, (0, \cdot, \cdot)) = 1$ then add tx to $\text{buffer}^{\text{PoS}}$.
 2. Execute $\text{Pstatus}_{\text{PoS}}(t^{\text{PoS}} - \Delta)$ (Fig. 6.3) thus obtaining $\text{status}^{\text{PoS}}$.
 3. Create a PoS block by running $\text{BlockCreate}^{\text{PoS}}$ on input the stake distributions represented by $\text{status}^{\text{PoS}}$ and the transactions $\text{buffer}^{\text{PoS}}$ thus obtaining $(B^{\text{PoS}}, C^{\text{PoS}}, \text{cert}^{\text{PoS}})$. Let B_n^{PoS} be the most recent block added to $\text{chain}^{\text{PoS}}$.
 4. If for every $\text{addr} \in C^{\text{PoS}}$ $\text{checklead}^{\text{PoS}}(\text{status}^{\text{PoS}}, \text{seed}^{\text{PoS}}, \text{addr}, |\text{chain}^{\text{PoS}}| + 1) = 1$ and $\text{isvalidchain}^{\text{PoS}}(\text{chain}^{\text{PoS}} \| B_n^{\text{PoS}}) = 1$ and $\text{certify}^{\text{PoS}}(\text{status}^{\text{PoS}}, H(B^{\text{PoS}}), H(B_n^{\text{PoS}}), h, |\text{chain}^{\text{PoS}}| + 1, \text{cert}^{\text{PoS}}) = 1$ then continue as follows, else reject the block and stop running Π_{PoS} .
 5. Set $b^{\text{PoS}}[s] \leftarrow (B^{\text{PoS}}, C^{\text{PoS}})$, $\text{chain}^{\text{PoS}} \leftarrow \text{chain}^{\text{PoS}} \| B^{\text{PoS}}$, $\text{Chains}[t^{\text{PoS}}] \leftarrow \text{chain}^{\text{PoS}}$.
 6. Create the transaction $\text{tx} = (\text{REWARD}, C^{\text{PoS}}, t^{\text{PoS}}, \text{cert})$ and invoke the oracle $\mathcal{O}$ of Π_{PoR} with input tx (we refer to Fig. 6.2 for the description of the oracle).
 7. Diffuse in the network the latest generated PoR block by sending the message (NEW-BLOCK, $t^{\text{PoR}}, B^{\text{PoR}}$).
- The block generated at the end of the slot is defined by a virtual block B , that contains all the transactions included in the latest block added to $\text{chain}^{\text{PoR}}$. This block is added to $\text{chain}^{\text{PoR}}$.

Normal mode: liveness violation support and detection

- If a party has $\text{flag}^{\text{liv}} = 1$, and the transaction (ORACLE, LIVENESS-VIOLATION) has appeared on the PoS chain, then go into *recovery mode*.
- If a party has $\text{flag}^{\text{liv}} = 0$, and the transaction (ORACLE, LIVENESS-VIOLATION) has appeared on the PoS chain and there is a PoS transaction on the ledger $\text{tx} = (\text{BLOCK-REQUEST}, i)$, do the following (else do nothing).
 1. If on the PoS ledger there is a transaction $\text{tx} = (\text{BLOCK-REQUEST-RESPONSE}, i, B)$ with B being the i -th PoR block stored in b^{PoR} then do nothing, else create the transaction $\text{tx} = (\text{BLOCK-REQUEST-RESPONSE}, i, B)$, where B is the i -th block contained in b^{PoR} , and invoke $\text{TxSubmit}^{\text{PoS}}(\text{tx})$.
- Invoke the procedure $\text{CheckLiveness}()$ (see. Fig. 6.7).

Normal mode for users: accepting safe transactions

If a party p_j receives a transaction $\text{tx} = (\text{STANDARD}, v, \text{addr}_i, \text{addrs}, \text{id}_i, \sigma_i)$, it marks it as a safe transaction and include it in the set of safe transactions S_j if the following holds

- $\text{chain}^{\text{PoR}}$ has a block B^* that contains a transaction $\text{tx} = (\text{PLEDGE}, \text{addr}_i, \text{addr}_j, \text{id}_i, T, \sigma_i)$ with $T \geq t^{\text{PoS}} + \Delta$.
- There exists a block in $\text{Chains}^{\text{PoS}}[\tau]$, with $\tau \leq t^{\text{PoS}} + \Delta$ that contains a transaction (CHECKPOINT, (h, i, cert)) such that $H(B^* \| \text{dspor}) = h$.
- All the transactions with $\text{tx}' = (\text{STANDARD}, v, \text{addr}_i, \dots) \in S$ have been checkpointed in a block of the PoS chain generated in a slot $\tau \leq t^{\text{PoS}} - \Delta$.
- $\exists k$ such that $\text{addrs}[k] = \text{addr}_j$ and $\text{TreasuryAllocation}(\text{addr}_i, \text{status}^{\text{PoR}}, \text{chain}^{\text{PoR}}) \geq v[k]$ where $(\text{status}^{\text{PoR}}, \text{seed}^{\text{PoR}}) = \text{aux}[t^{\text{PoS}} - \Delta]$ and $\text{chain}^{\text{PoR}} \leftarrow \text{Chains}[t^{\text{PoS}} - \Delta]$.

Normal mode for users: GetChain

Return $\text{chain}^{\text{PoR}}$.

Figure 6.2: The Oracles $\mathcal{O}$ and $\mathcal{O}^{\text{liv}}$

- The oracle $\mathcal{O}$ has read access to the PoS and the PoR chains, and write access to the PoR chain. On input $\text{tx} = (\text{REWARD}, C^{\text{PoS}}, t^{\text{PoS}}, \text{cert})$ returns 1 if the following conditions hold (and return 0 otherwise).
 - C^{PoS} contains the set of the addresses of the leaders that generated the PoS block in slot t^{PoS} , and these addresses have not received any reward yet.
 - The PoS block generated in the t^{PoS} -th slot contains only valid checkpoints for the PoR chain.

- The oracle $\mathcal{O}^{\text{liv}}$ has read access to the PoR chain, and write access to the PoS chain. On input $\text{tx} = (\text{LIVENESS-VIOLATION})$ returns 1 if no honest parties receive a valid PoR block slot i with $i \in \mathbb{N}$

Figure 6.3: The protocol $\text{Pstatus}^{\text{PoS}}(k)$

Invoke $\text{GetChain}^{\text{PoR}}$ thus obtaining $\text{chain}^{\text{PoR}}$.
 Invoke $\text{GetChain}^{\text{PoS}}$ thus obtaining $\text{chain}^{\text{PoS}}$ and let B^{PoS} be the latest block of $\text{chain}^{\text{PoS}}$ generated at (or before) the slot k containing the valid transaction $\text{tx} = ((\text{CHECK-POINT}, (h_i, i, \text{cert}_i)), ((\text{CHECK-POINT}, (h_{i+1}, i + 1, \text{cert}_{i+1})), \dots, (\text{CHECK-POINT}, (h_{i+j}, i + j, \text{cert}_{i+j})))$.
 Let $\text{subchain} = (B_0, \dots, B_i, B'_{i+1}, \dots, B'_{i+j})$ the first $(i + j)$ blocks of $\text{chain}^{\text{PoR}}$. If $(H(B'_{i+1}), \dots, H(B'_{i+j})) = (h_{i+1}, \dots, h_{i+j})$ then do the following, else return^a.
 Parse $(B_0, \dots, B_i, B'_{i+1}, \dots, B'_{i+j})$ and for every address compute the amount of stake available, thus obtaining the tuples $\{\text{addr}_k, \mathbf{S}_k\}_{k \in [m]}$, where $\mathbf{S}_k$ denotes the balance of the address addr_k , and m is the total number of addresses in subchain .
 Return $\{\text{addr}_k, \mathbf{S}_k\}_{k \in [m]}$.

^a In this case *recovery* it has been, or it will be invoked by the honest parties.

Our protocol has two main modes of operation: *normal* and *recovery* mode. At a high level, in *normal mode* (see Fig. 6.1 for a formal description) each party will run the two protocols Π_{PoR} and Π_{PoS} using Π_{PoR} as the primary chain (where the transactions will be posted), and using Π_{PoS} *only* to checkpoint the hash of the block generated via Π_{PoR} . In more detail, any time that a block B is generated in the PoR chain (let us say that B is the k -th block), the maintainers issue a transaction $(\text{CHECK-POINT}, H(B), j, \text{cert})$ (recall that cert is used in the secondary chain to certify that $H(B)$ does correspond to a hash of an actual block of the PoR chain).

Fork detection. If any party p_j executing the protocol notices that the hash of, for example, the k -th block in his PoR chain differs from the hash checkpointed on the PoS chain, then this party creates the transaction $\text{tx} = (\text{COMPLAINT}, h^*, \text{cert}^*, j)$, where h^* is the hash of the block in the local chain of p_j , and cert^* is the certificate p_j obtained when the k -th block of the PoR chain was generated by running $\text{BlockCreate}^{\text{PoR}}$. Then p_j submits tx via $\text{TxSubmit}^{\text{PoS}}(\text{tx})$.

Once tx is accepted by the PoS chain, then all the parties running our protocol will notice that the PoR chain had a fork, hence, they will enter in what we call *Recovery Mode*. In this mode, the parties will discard all the PoR blocks that have been checkpointed after the slot $\Delta = 2\text{liv}^{\text{PoS}}$. This guarantees that the chopped PoR chain is a chain agreed upon by all the honest parties, and this is for the following reasons. Let us say that at time t an honest party noticed that his local PoS chain was inconsistent with the checkpoint (i.e., it noticed a fork). This means that there could have been something wrong with the PoR chain already liv^{PoS} slots earlier (this is because in the worst case, an honest party checkpoint is included after liv^{PoS} slots). Moreover, once a fork is detected, the complaint can take up to liv^{PoS} slots. Hence, chopping the PoR chain of all the blocks generated in the latest $\Delta = 2\text{liv}^{\text{PoS}}$ slots guarantees that all the parties in recovery mode agree with the current view of the PoR chain. We note that we can use cert and cert^* to eventually identify (malicious) block leaders that accepted different block headers for the same slot. This information can be used to put to 0 the reputation of these parties. For the formal description of how the forks are detected, we refer to Fig. 6.4. We now elaborate more on the action performed during the recovery-mode phase.

Figure 6.4: Fork detection

Check for PoR forks in the PoR chain

In every PoS slot t^{PoS} , each party p_i does the following.

- Let B^{PoS} be the latest PoS that contains transactions $\text{tx} = ((\text{CHECK-POINT}, (h_i, i, \text{cert}_i)), ((\text{CHECK-POINT}, (h_i, i + 1, \text{cert}_{i+1})), \dots, (\text{CHECK-POINT}, (h_{i+j}, i + j, \text{cert}_{i+j})))$
- Let $B_i, \dots, B_{i+j}$ be the sequence of blocks included in $\text{chain}^{\text{PoR}}$ starting from the index i until the index $i + j$. Let $k \in [1, \dots, j]$ be the smallest integer such that s.t $H(B_{i+k}) \neq h_{i+k}$. If such a k does not exist

then continue to the next step, else create the transaction $\text{tx}' = (\text{COMPLAINT}, H(B_{i+k}), \text{cert}_{i+k}, i+k)$, invoke $\text{TxSubmit}^{\text{PoS}}(\text{tx}')$ and go to **Recovery Mode**.

- Parse B^{PoS} to retrieve the transactions $\text{tx} = (\text{COMPLAINT}, h^*, \text{cert}^*)$. If such a transaction does not exist, then conclude the protocol round, else continue as follows.
- If $\text{certify}^{\text{PoR}}(\text{status}_{k-1}^{\text{PoR}}, H(B_{k-1}^{\text{PoR}}), h^*, k, \text{cert}^*) = 1$ and $H(B_k^{\text{PoR}}) \neq h^*$ then go to **Recovery Mode** else conclude the protocol round.

Figure 6.5: Recovery mode

Recovery mode initialization. Every party in Recovery Mode does the following steps

- Set $\tau^{\text{PoS}} = t^{\text{PoS}} - \Delta$, and $\tau^{\text{PoR}} = \ell \cdot (\tau^{\text{PoS}} - 1)$.
- Get $(\text{status}^{\text{PoR}}, \text{seed}^{\text{PoR}}) \leftarrow \text{aux}[\tau^{\text{PoR}}]$ and $(B^{\text{PoR}}, C^{\text{PoR}}) \leftarrow b^{\text{PoR}}[\tau^{\text{PoR}}]$.
- Set $\text{fork} \leftarrow \text{chain}^{\text{PoR}}$, and redefine $\text{chain}^{\text{PoR}}$ as all the blocks of the current PoR chain from the genesis block, until the block B^{PoR} (the latest safely checkpointed block on $\text{chain}^{\text{PoS}}$).
- Remove from $\text{chain}^{\text{PoR}}$ all the transactions contained in PoR blocks generated after the slot τ^{PoR} .

Recovery mode. After the initialization, in every new PoS epoch t^{PoS} each party, on input, a set of transactions TxS^{PoS} , does the following.

- For every $\text{tx} \in \text{TxS}^{\text{PoS}}$, if $\text{validity}^{\text{PoS}}(\text{tx}, \text{chain}^{\text{PoS}}, \text{buffer}^{\text{PoS}}, (1, \text{chain}^{\text{PoR}}), \tau^{\text{PoR}}) = 1$ then add tx to $\text{buffer}^{\text{PoS}}$.^a
- Execute $\text{Pstatus}^{\text{PoS}}(\tau^{\text{PoS}})$ thus obtaining $\text{status}^{\text{PoS}}$.
- Create a PoS block by running $\text{BlockCreate}^{\text{PoS}}$ on input the stake distributions represented by $\text{status}^{\text{PoS}}$ and the transactions $\text{buffer}^{\text{PoS}}$ thus obtaining $(B^{\text{PoS}}, C^{\text{PoS}}, \text{cert}^{\text{PoS}})$. Let B_n^{PoS} be the most recent block added to $\text{chain}^{\text{PoS}}$.
- If for every $\text{addr} \in C^{\text{PoR}}$ $\text{checklead}^{\text{PoR}}(\text{status}^{\text{PoR}}, \text{seed}^{\text{PoS}}, \text{addr}, |\text{chain}^{\text{PoS}}| + 1) = 1$ and $\text{isvalidchain}^{\text{PoS}}(\text{chain}^{\text{PoS}} || B^{\text{PoS}}) = 1$ and $\text{certify}^{\text{PoS}}(\text{status}^{\text{PoS}}, H(B^{\text{PoS}}), H(B_n^{\text{PoS}}), h, |\text{chain}^{\text{PoS}}| + 1, \text{cert}^{\text{PoS}}) = 1$ then continue as follows, else reject the block and stop.
- Set $b^{\text{PoS}}[s] \leftarrow (B^{\text{PoS}}, C^{\text{PoS}})$, and $\text{chain}^{\text{PoS}} \leftarrow \text{chain}^{\text{PoS}} || B^{\text{PoS}}$.
- The block generated at the end of the slot is defined by a virtual block B , that contains all the transactions included in the latest block added to $\text{chain}^{\text{PoS}}$.^b This block is added to $\text{chain}^{\text{PoR}}$.

Recovery mode: GetChain

Return $\text{chain}^{\text{PoR}}$.

^a Note that the validity predicate now accepts PoR-type of transactions, hence, the PoS chain from this point on will contain and validate PoR transactions.

^b Note that now the local chain of the parties is extended by using the transactions included in the PoS chain, and not those included in the PoR chain.

Recovery mode. During Recovery Mode (that we formally describe in Fig. 6.5) the main goal of the protocol is to refund parties that were victims of double spending (due to the forks happening) while still allowing parties to post new transactions, but at the speed of the secondary chain (the PoS chain). To explain how we refund parties that are victims of double spending in the recovery mode, we need to introduce the type of transactions accepted in our system. In our protocol, we distinguish between two types of transactions: *standard* and *safe* transactions. A party p_i with address addr_i , to issue a standard transaction and pay a party p_j some amount of coins v , creates the transaction $\text{tx} = (\text{STANDARD}, v, \text{addr}_i, \{\text{addr}_j\}, \text{id}, \sigma)$, where id is a unique identifier, and σ is the signature of the transaction for the secret key addr_i . A sender can also specify a vector of receivers and a vector of coins to pay multiple parties via a single transaction. Then p_i invokes $\text{TxSubmit}^{\text{PoR}}(\text{tx})$ (recall that $\text{TxSubmit}^{\text{PoR}}$ is the protocol that a party can use to submit a transaction to the PoR chain). Once the receiver (with address addr_j) sees tx appearing in his local chain, then it declares the transaction as valid (recall that PoR has instant finality). This is the most basic type of transaction that a blockchain protocol could offer. However, if a fork happens, then a malicious p_i can spend more than he has actually available by creating a fork (this attack is often referred to as *double spending*). We have argued how our protocol can detect a fork (using the PoS chain), and let parties agree on a common view of the PoR (pre-fork), and continue running the protocol on the PoS chain.

In Sec. 7, we describe two sources of funds to refund the victims of double spending. The first is simple: we can require that if a party wishes their transaction to be instantly confirmed, they must have pledged a sufficient amount of collateral towards the transaction receiver (the coins pledged will

then be locked). The second method is to use system funds (called the treasury) to pay back victims of double spending, which we detail the implications of in Sec. 7. Intuitively, when p_i wants to pay p_j , they issue a standard transaction $\text{tx} = (\text{STANDARD}, v, \text{addr}_i, \{\text{addr}_j\}, \text{id}, \sigma)$. Then, p_j accepts the payment if it appears on a PoR block and if there is a sufficient pledged collateral to pay for tx in the event of a fork. Such a pledge is captured by transaction $\text{tx}^* = (\text{PLEDGE}, \text{addr}_i, \text{addr}_j, T)$, where T is an expiry timer, which must be checkpointed on the PoS chain. If a conflicting transaction indeed is confirmed, a fork (and double spending) happens, and the system goes into Recovery Mode, then p_j creates a *payback* transaction $\text{tx}' = (\text{PAYBACK}, \text{addr}_j, \text{tx}, \sigma)$ to submit it on the PoS chain by invoking $\text{TxSubmit}^{\text{PoS}}(\text{tx}')$ (recall that $\text{TxSubmit}^{\text{PoS}}$ is the protocol that a party can use to submit a transaction to the PoS chain). The PoS chain will accept this transaction now, and it will use the locked coins to pay p_j . We note that if no fork happens, p_i can keep issuing safe transactions to p_j as long as the timer in tx^* has not expired. Moreover, p_j , upon seeing tx part of his local chain, it can immediately assume (at the speed of the PoR chain) that he will receive the v coins (recall that tx was a payment of v coins). p_j just needs to be careful to only accept unconfirmed transactions whose total amount does not exceed the pledged amount. For more detail on how these transactions are managed, we refer to the part of Fig. 6.1 that we named “Normal mode for users: accepting safe transactions”.

Finalizing the recovery. The Recovery Mode lasts a fixed number of rounds that we denote by rectime . After that, the maintainers will try to restart the PoR chain. This is done by computing the balances of all the parties using: 1) the transactions included in the pre-fork PoR chain, 2) the standard transactions that have been included in the PoS chain in recovery mode, and 3) the transactions $(\text{PAYBACK}, \cdot)$ (included in the PoS chain as well). For every balance balance_i associated to a party p_i , each maintainer creates a special transaction $\text{tx}_i = (0, \text{addr}_i, \text{balance}_i)$. Then it will detect the parties that tried to double spend. This can be easily done as the balances of these parties will be less than 0. This is because every receiver will submit a transaction of the type $(\text{PAYBACK}, \cdot, \text{tx})$, where tx is an attempt of double spending from some corrupted sender p_i . When computing the balances, we consider all the transaction tx wrapped in *payback* transactions, hence, the balance of a party that tried to double spend will necessarily become less than 0, unless he paid his debt via a standard transaction during the recovery mode. For all the parties that have been detected double spending, the maintainers set their reputation to 0 thus obtaining a new reputation vector $\text{status}^{\text{PoR}}$. Then each maintainer creates a transaction $\text{tx}^* = (\text{REPUTATION}, \text{status}^{\text{PoR}})$, creates a new genesis block $\mathcal{G}^{\text{PoR}}$ that includes the transactions $(\text{tx}^*, \{\text{tx}_i\}_i)$, and the seed computed by hashing the latest block generated in the PoS chain. Then checkpoint $h \leftarrow H(\mathcal{G}^{\text{PoR}})$ on the PoS chain (by submitting a transaction $(\text{CHECKPOINT}, h, 0)$ via $\text{TxSubmit}^{\text{PoS}}$). Each party waits liv^{PoS} slots, to make sure that all the parties have the new genesis block checkpointed, and go in normal mode, using the new genesis block to run the PoR protocol from scratch. For the formal description of how this part of the protocol works, we refer to Fig. 6.6.

Figure 6.6: Recovery finalization

After a party p_i has been in Recovery Mode for rectime , each party p_i stops accepting new transactions for liv^{PoS} slots. After this, in every slot t^{PoS} p_i does the following steps.

- Let chain^* be the PoS chain at the end of slot $t^{\text{PoS}} - 1$, and let history be the set of all the transactions of the form $(\text{PAYBACK}, \cdot), (\text{STANDARD}, \cdot) \in \text{chain}^*$. Let addr_i, S_i denote the balance of party p_i in $\text{chain}^{\text{PoR}}$.
- For every transaction $(\text{PAYBACK}, \text{addr}_j, \text{tx}, \cdot) \in \text{history}$ and for every $\text{tx} \in \text{history}$ with $\text{tx} = (\text{STANDARD}, \cdot)$ do the following
 - Parse tx as $(\text{STANDARD}, v, \text{addr}_i, \text{addrs}, \text{id}_i, \sigma_i)$
 - Compute $\text{spent}_i \leftarrow \text{spent}_i + \sum_k v[k]$.
 - For each k such that $\text{addrs}[k] = \text{addr}_i$, compute $\text{received}_i \leftarrow \text{received}_i + v[k]$.
- $\forall i$ such $S_i - \text{spent}_i < 0$ add addr_i to the set DS .
- Get $\text{status}^{\text{PoR}}$ from $\text{Pstatus}^{\text{PoR}}(\tau^{\text{PoR}})$ and modify the reputation vector $\text{status}^{\text{PoR}}$ by setting to 0 the reputation of all the parties in DS .
- Define the transaction $\text{tx}^* = (\text{SET-REPUTATION}, \text{status}^{\text{PoR}})$
- $\forall i$ define the transaction $\text{tx}_i = (0^\lambda, \text{addr}_i, \text{balance} - \text{spent} + \text{received})$
- Compute $\text{seed}^{\text{PoR}} \leftarrow H(\text{GENESIS}, B || \text{dspor})$ where B is the most recent block generated in the slot interval $[0, \dots, t^{\text{PoS}} - \text{co}^{\text{PoS}}]$.
- Define the genesis block $\mathcal{G}^{\text{PoR}}$, containing seed^{PoR} , and the transactions $(\text{tx}^*, \{\text{tx}_i\}_i)$.
- Invoke $\text{TxSubmit}^{\text{PoS}}((\text{CHECK-POINT}, H(\mathcal{G}^{\text{PoR}}), 0, \perp))$

- Do not accept any transaction for liv^{PoS} slots, and after that, start running the protocol in normal mode (Fig. 6.1) using $\mathcal{G}^{\text{PoR}}$ as the genesis block for the PoR chain.

Transaction Liveness Violation. We refer to the introductory part of the paper for an informal description of how we guarantee transaction liveness. We refer to Fig. 6.1 and Fig. 6.7 for a formal description of the protocol.

We have not elaborated on how the rewards for the PoS chain are generated (i.e., exported to the primary chain). We now provide a high-level description of this mechanism. When in normal mode, our system only uses the primary chain (Π^{PoR}) to manage all the transactions related to *money movement*. For this reason, it is also easy to reward block leaders of the PoR, as we can simply use whatever mechanism is embedded in Π^{PoR} . To reward the block leaders of the PoS chain, it requires a bit of work. We want to issue rewards that can (of course) be spent. Hence, any reward generated on the PoS chain must first be exported on the PoS chain in order to be spent. One way to do that would be to just use the reward mechanism of the PoS chain (hence, using coins unrelated to the PoR chain), and later export these coins in the PoR chain, to convert them into coins that can actually be spent on the PoR chain. This approach, unfortunately, has a major issue. In the event the PoS chain is broken, an adversary could generate unlimited PoS coins and redeem them to the PoS chain, effectively stealing money from the system. To avoid this type of issue, in normal mode, only the PoR chain can generate new coins. To reward PoS block leaders, we instead proceed as follows. We first define an oracle, that on input $\text{tx} = (\text{REWARD}, C, m, \text{cert})$, returns 1 if on the PoS chain, the block with index m contains valid PoR checkpoints, and the leaders who generated the PoS block are all included in C .

Property 5.1 of the PoR chain guarantees that when ASP^{PoR} holds and all the honest parties have the output of the oracle, then the correct parties will be rewarded. The final question to answer is how to realize such an oracle. This is trivial, as every party can emulate the behaviour of the oracle by looking at the PoS and PoR chain, and check if a block was correctly checkpointed by a certain set of parties.

Figure 6.7: Transaction submission and *transaction* liveness violation

When a party p_i wants to submit a transaction with content m (e.g., $m = (\text{STANDARD}, v, \text{addr}_i, \text{addr}_j, \text{id})$) it computes $\sigma \leftarrow \text{Sign}(\text{sk}, m)$, sets $\text{tx} = (m, \sigma)$ and invokes $\text{TxSubmit}^{\text{PoS}}((m, \sigma))$. If the transaction does not appear in the local $\text{chain}^{\text{PoR}}$ of p_i within liv^{PoR} slots, then set $\text{tx}' = (\text{LIVENESS-ISSUE}, \text{tx})$, compute $\sigma' \leftarrow \text{Sign}(\text{sk}, \text{tx}')$ and invoke $\text{TxSubmit}^{\text{PoS}}(\text{tx}')$.
 $\text{CheckLiveness}()$

Check for liveness violation on PoR

Every maintainer p_i does the following

- Checks if a new transaction $\text{tx} = (\text{LIVENESS}, \text{tx}', \sigma')$ has been added to $\text{chain}^{\text{PoS}}$, but it does not appear on $\text{chain}^{\text{PoR}}$. If this is the case then invoke $\text{TxSubmit}^{\text{PoR}}(\text{tx})$. If after liv^{PoS} slots tx appears on $\text{chain}^{\text{PoR}}$ then return, else enter into **Recovery Mode**.^a
- If the transaction LIVENESS-VIOLATION has appeared on the PoS and there exists $i < t^{\text{PoS}}$ such that all following occur, then go into **Recovery Mode**.
 1. $b^{\text{PoR}}[i] = \perp$
 2. A transaction $\text{tx} = (\text{BLOCK-REQUEST}, i)$ has been confirmed on the PoS chain at a certain slot t^* .
 3. No valid transaction $\text{tx} = (\text{BLOCK-REQUEST-RESPONSE}, i, B)$ has been confirmed on the PoS chain
 4. $t^{\text{PoR}} - t^* \geq \text{liv}^{\text{PoS}}$ (i.e., despite having enough time, nobody submitted a transaction BLOCK-REQUEST-RESPONSE for the i -th block, hence, nobody has it).

^a Observe that if a transaction does not appear in $\text{chain}^{\text{PoR}}$ according to the liveness parameter, then it must be that the PoR chain is compromised. Given that we assume that only one chain can be compromised, this means that parties can safely rely on the PoS chain. At this point, either the parties agree that tx does not appear on PoR, and hence go into recovery mode, or some parties agree and some others disagree. This means that there is a fork, hence, all the honest parties go into recovery mode.

Formal Statements. Below we state the formal theorems that we prove regarding our protocol, and refer to Appendix B.2 for their security proofs.

Theorem 6.1 (Safe PoR, Compromised PoS). *If $\text{ASP}^{\text{PoR}}(t) = 1$ for every $t \in [0, \dots, m]$, then at any time slot $i \in [0, \dots, m]$ Π enjoys safety, liveness, and transaction liveness of the underlying PoR chain.*

Theorem 6.2 (Safety: Compromised PoR, Safe PoS). *If $\text{ASP}^{\text{PoS}}(t) = 1$ for every $t \in [0, \dots, m]$ then Π enjoys safety: at any time slot $i \in [0, \dots, m]$, the local chains held by any two honest parties are consistent if they each ignore the blocks generated in the latest $D = \max\{2\text{liv}^{\text{PoS}}, \text{co}^{\text{PoS}}\}$ PoS slots.*

Theorem 6.3 (Liveness: Compromised PoR, Safe PoS). *If $\text{ASP}^{\text{PoS}}(t) = 1$ for all $t \in \mathbb{N}$, then a transaction tx submitted by an honest party is included in the honest parties' local chains within $3\text{liv}^{\text{PoS}} + \text{co}^{\text{PoS}}$ PoS slots.*

Theorem 6.4 (Recovery: Compromised PoR, Safe PoS). *If $\text{ASP}^{\text{PoR}}(t) = 1$ for every $t \geq t_1$ (for arbitrary $t_1 \in \mathbb{N}$) and $\forall k \text{ASP}^{\text{PoS}}(k) = 1$ and Π^{PoR} enjoys the property of seed recoverable security, then let τ be the time at which a transaction tx is submitted by an honest party. If $\tau \geq t_1 + \text{rectime} + 4\text{liv}^{\text{PoS}} + \text{co}^{\text{PoS}}$ then tx is included in the honest parties' local chains within liv^{PoR} PoR slots. Moreover, after the time slot τ (again with $\tau \geq t_1 + \text{rectime} + 4\text{liv}^{\text{PoS}} + \text{co}^{\text{PoS}}$), the safety of Π holds with respect to the parameter co^{PoR}*

Theorem 6.5 (Payment Finality). *If an honest party p accepts a safe transaction of value v in the slot τ and $\forall k \text{ASP}^{\text{PoS}}(k) = 1$, and $\exists t$ such that $\text{ASP}^{\text{PoR}}(t) = 0$ then the stake represented by the chain of all honest parties in the slot $(t_1 + 4\text{liv}^{\text{PoS}})$ will reflect the fact that the balance of p has increased by v .*

7 Collaterals for Fast Payments

Recall that to achieve *safe* transactions that can be accepted by a receiver p_j instantly, the sender must *pledge* an amount of coins to the receiver, which will be used to pay the receiver in case of a double-spend. We describe two sources for the pledged coins (which can be used in conjunction with each other): (1) The sender p_i can deposit collaterals using their own coins (which we formalise in App. C). (2) We design a system that manages a pot of funds called the *treasury* used for pledging, where the amount allocated to each pledge is proportional to (a function of) the pledger's reputation (formalised in Fig. 7.1). As is with the rest of the paper, $\Delta \leftarrow 2\text{liv}^{\text{PoS}}$, where liv^{PoS} is the PoS's liveness and Δ is the number of slots to finalize a PoR block in the hybrid protocol. See App. C for discussions and proofs.

7.1 Fast Payments without Depositing Collateral

We leverage reputation in PoR as a 'credit-score' to allocate system funds (which we call the *treasury*) to parties as the collateral. We define the following operations on the treasury.

Initialization: We hard-code a starting amount T_0 in the treasury (e.g., $T_0 = 0$).

- On $(\text{CheckTreasury}, \text{chain})$: Returns the amount of coins in the treasury according to PoR chain chain at latest slot t . The amount returned is T_0 plus the total inflow to the treasury, minus the total outflow from the treasury.
The total *outflow* is the sum of all amounts in payback transactions (i.e., those labeled PAYBACK). Similarly the total *inflow* is the total amount of coins entering the treasury (e.g., through transaction fees, as a percentage of block rewards, etc.—for generality here we don't specify a particular mechanism for inflow).
- On $(\text{CheckAllocation}, \text{chain}, p, \Delta)$: Returns the amount allocated to party p from the treasury, according to PoR chain chain . This returns the output of $\text{TreasuryAllocation}_\Delta(p, \text{chain})$ (Fig. 7.1 in Appendix). Honest parties will call this operation with $\Delta = 2 \cdot \text{liv}^{\text{PoS}}$.

Accepting payments using collateral from the treasury. Suppose a party p_j receives a transaction from p_i to p_j at slot t . Then p_j can accept this as a safe transaction if it appears on a PoR block and (1) a PLEDGE is checkpointed in the hybrid protocol (Δ slots have passed without a PoR fork detected), and (2) the total amount in accepted, not-yet-checkpointed transactions from p_i to p_j would not exceed the allocated collateral from the treasury. Formally, for (2) p_j verifies that, if p_j accepts this transaction,

then the total amount in accepted transactions that are from p_i to p_j , between slots $t - \Delta$ and t , is at most $(\text{CheckAllocation}, \text{chain}, p_i, \Delta)$. Here chain is p_i 's PoR chain at slot t .

Recovery phase and payback using collateral from the treasury. Suppose a party sees a PAYBACK transaction (which is processed on the PoS chain during recovery phase) claiming p_i owes v coins to p_j . This transaction is valid if we observe on the PoS chain: (1) two conflicting transactions that prove p_j is the victim of double-spending by p_i for amount v , (2) p_i pledged to p_j , and (3) $(\text{CheckAllocation}, \text{chain}, p, \Delta) \geq v$, where chain is the most recent PoR chain in the honest party's view before the recovery phase.

Figure 7.1: Computing the amount from treasury allocated to a party

TreasuryAllocation $_{\Delta}(p, \text{chain})$

Parameter/Input/Output: TreasuryAllocation is parametrized by Δ . The algorithm returns the amount of the treasury allocated to party p , according to the PoR chain chain .

1. Let t be the current slot, and $t' \leftarrow t - \Delta$. If the treasury has not been created at slot t' yet, return 0.
2. Let $\text{Rep} = (r_1, \dots, r_n)$ be the reputation vector according to chain at t' .
3. Let r be p 's reputation in Rep .
4. Let $T \leftarrow$ the amount of coins held in the treasury up to slot t' of chain (calculated using CheckTreasury).
5. $\ell \leftarrow \left\lfloor T \cdot \frac{\Pr(I_{t',r}^{\text{PoR}})}{m} \right\rfloor$. Where: $\Pr(I_{t',r}^{\text{PoR}})$ is the probability for a party with reputation r is chosen as a PoR block creator according to reputation vector Rep defined above, and $m = \sum_{i \in [n]} \Pr(I_{t',r_i}^{\text{PoR}})$ is a constant, equal to the number of PoR block creators chosen by the PoR lottery per block.
6. return ℓ

Defining and Proving Desiderata for Treasury Allocation We list and prove the following natural desiderata for a system which allocates system funds (a treasury) to parties, under both static and dynamic reputation systems.

- **Allocation agreement.** For any party p and PoR chains chain and chain' held by any two honest parties at some slot t , $(\text{CheckAllocation}, \text{chain}, p, \Delta) = (\text{CheckAllocation}, \text{chain}', p, \Delta)$
- **Allocation persistence.** Let chain_1 be the PoR chain held by some honest party at slot t_1 and chain_2 be the PoR chain held by some honest party at slot $t_2 > t_1$. Then unless all honest parties end recovery phase at some $t \in [t_1, t_2]$, $(\text{CheckAllocation}, \text{chain}_1, p, \Delta) \geq (\text{CheckAllocation}, \text{chain}_2, p, \Delta)$
- **Treasury always sufficient.** The value returned by $(\text{CheckTreasury}, \text{chain})$ for a PoR chain held by an honest party's view will be non-negative.
- **Fair allocation.** If current slot is t and parties p_i and p_j both have the same reputation according to chain at $t - \Delta$, then $(\text{CheckAllocation}, \text{chain}, p_i, \Delta) = (\text{CheckAllocation}, \text{chain}, p_j, \Delta)$.
- **No incentives to misbehave for a period of Δt .** This property holds if, for any pair p_h and p_a with the same stake and reputation, where p_h is an honest party and p_a a dishonest party who had their reputation zeroed out (which would be observed by all honest parties during recovery) at some time t_a , the utility $u_{t_a, \Delta t}(p_a) < u_{t_a, \Delta t}(p_h)$. The utility u is defined below:

$$u_{t_a, \Delta t}(p) = \sum_{t \in [t_a, t_a + \Delta t]} \text{rew}_{\text{PoR}} \cdot \Pr(I_{t, r_t}^{\text{PoR}}) + \sum_{t \in [t_a, t_a + \Delta t]} \text{rew}_{\text{PoS}} \cdot \Pr(I_{t, s_t}^{\text{PoS}}) + \text{fprofit} \cdot \Pr(D)$$

Here, I_{t, r_t}^{PoR} is the event where p has reputation r_t at slot t and is chosen as a PoR block creator for slot t by an honest party in the hybrid protocol (and rew_{PoR} is the block reward in PoR); I_{t, s_t}^{PoS} is the analogous event for the PoS where s_t is the stake of party p at slot t (and rew_{PoS} is the block reward in PoS). $\Pr(D)$ is the probability that p has double-spent (make another party p' accept a transaction which is later found to be double-spending), and fprofit is the amount of coins gained by p by double-spending. **Informally**, this says that over a period of Δt , the honest party's total expected utility from PoR and PoS block rewards (honest parties don't double-spend), is greater than the expected utility of the adversarial party.

In particular, the first three properties imply that collateral from the treasury can be treated in the same way as collateral pledged/locked using a party's own funds.

Lemma 7.1 (Proof in App. C). *Using a treasury scheme that is always sufficient, and has allocation persistence and agreement, the hybrid protocol achieves payment finality.*

7.2 Case 1: Static Reputation System.

We first prove the above desiderata in a static reputation system, for the operations we defined on our treasury. This is the case when reputation and number of parties do not change, except the reputation of misbehaving parties are zero'd out during recovery phase, after all double-spent transactions are paid back.

Intuitively, our allocation algorithm `TreasuryAllocation` (Fig. 7.1) is fair—it allocates the treasury proportionally to a function of the reputation. Moreover, it allocates based only on the checkpointed blocks in the PoR chain (so honest parties agree on the allocation, whose amount persists until the end of recovery phase where all double spending transactions are already resolved). For ‘no incentives to misbehave’, we require an extra assumption that the amount in the treasury (and thus the amount allocated to each party) is not too large, to prevent incentives to double-spend.

Theorem 7.1 (Proof in App. C). *For the treasury specified in Sec. 7.1, under a static reputation system: allocation agreement, allocation persistence, treasury always sufficient, and fair allocation are satisfied with overwhelming probability. Moreover, if for any Δt , $(\text{CheckTreasury}, \text{chain}) < m \cdot \Delta t \cdot \text{rew}_{\text{PoR}}$ for any chain held by an honest party (and m is the number of parties chosen by the PoR lottery), there are no incentives to misbehave for Δt .*

We defer the discussion on dynamic reputation to Sec. 7.3.

7.3 Case 2: Dynamic Reputation System.

Realistically, we would like parties’ reputation increase when they participate correctly, and new parties to join. In a dynamic reputation system, we start with an initial reputation vector (for an initial set of parties), after which new parties can join (when a new party enters the system, we’ll assume their reputation starts at zero for simplicity), and parties can have their reputation increased, stay the same, or be zeroed (e.g., for misbehaving).

Most of the desiderata in our proof of Thm. 7.1 do not depend on the static-ness of the reputation system. The only slight caveat is that for the same incentives proof, we require that within Δt slots after recovery, no new parties join and parties with zeroed out reputation cannot increase their reputation. The new challenge is achieving *allocation persistence* and *fair allocation* in combination. Specifically, if new parties enter and increase their reputation, the treasury inflow must keep up with the additional allocations needed to given to these new parties (for fair allocation). Otherwise, some existing allocations must necessarily decrease, which can break allocation persistence. Our additional assumption is then (intuitively), that the *treasury grows proportionally* to the growth of reputation in the system.

Theorem (Informal, see Thm. 7.2) Under dynamic reputation, the scheme in Sec. 7.1 satisfies *allocation agreement*, *treasury always sufficient*, and *fair allocation*. Moreover, under assumptions on the growth of the treasury and growth of parties’ reputations, *allocation persistence* holds and there is *no incentives to misbehave* for Δt , for any Δt .

In a dynamic reputation system, if the treasury remains static, then new parties joining and gaining reputation makes it impossible to satisfy both the properties of *fair allocation* and *allocation persistence*. This is because there may be no sufficient treasury to allocate to these new parties, without decreasing the allocation of existing parties. To combat this, we must assume a growth in the treasury, as a function of the growth of reputation in the system. We capture the decrease (for parties who retained or increased their reputation) in the probability of being chosen by the PoR lottery, from slot t_1 to t_2 , by $\text{maxReduction}_{t_1, t_2}$. Formally: let $\Pr(I_{\text{Rep}}^r)$ be the probability of a party with reputation r is selected as a PoR block creator when the reputation vector is Rep . For any slots $t_1 > \Delta$ and $t_2 > t_1$, let: $\text{Rep}_1, \text{Rep}_2$ be the reputation vectors and T_1, T_2 be the amount in the treasury, for slots $t_1 - \Delta$ and $t_2 - \Delta$ respectively (for both the reputation vector and treasury amount). Then we define

$$\text{maxReduction}_{t_1, t_2} := \max_{i: r_i \in \text{Rep}_1, r'_i \in \text{Rep}_2, r'_i \geq r_i} \frac{\Pr(I_{\text{Rep}_1}^{r_i})}{\Pr(I_{\text{Rep}_2}^{r'_i})}$$

Theorem 7.2. *For the treasury specified in Sec. 7.1, under a dynamic reputation system: allocation agreement, treasury always sufficient, and fair allocation are satisfied with overwhelming probability.*

Moreover, for any $t > \Delta$, let T_t be the treasury according by (any) honest party at slot $t - \Delta$, then allocation persistence also holds if for all t , we have $T_{t+1} \geq T_t \cdot \text{maxReduction}_{t,t+1}$.

Further, if for any Δt , no new parties are introduced in the system and parties with zeroed out reputation cannot increase their reputation for Δt slots after recovery phase, and the treasury $(\text{CheckTreasury}, \text{chain}) < m \cdot \Delta t \cdot \text{rew}_{\text{PoR}}$ for any chain held by an honest party (and m is the number of parties chosen by the PoR lottery), there are no incentives to misbehave for Δt .

Proof (Proof: Thm. 7.2). The proofs for allocation agreement, treasury always sufficient, and fair allocation are exactly the same as in the proof for the static reputation case.

For no incentives to misbehave, the only difference is that we further assume parties with zeroed out reputation cannot increase their reputation for Δt slots after recovery phase, so that those parties cannot obtain PoR block rewards during these slots. Moreover, since no new parties are introduced for Δt slots, the probability of honest party p_h creating a PoR block cannot decrease during this time.

For allocation persistence: we show that for all slots t , the allocation to a party p cannot decrease if p 's reputation does not decrease. From the definition the `TreasuryAllocation` algorithm, we see our goal

is to show $T_{t+1} \cdot \frac{\Pr(I_{\text{Rep}_2}^{r'_i})}{m} \geq T_t \cdot \frac{\Pr(I_{\text{Rep}_1}^{r_i})}{m}$, where $\text{Rep}_1, \text{Rep}_2$ be the reputation vectors and T_1, T_2 be the amount in the treasury, for slots $t - \Delta$ and $t + 1 - \Delta$ respectively (for both the reputation vector and treasury amount). This holds given the assumption in our theorem.

Acknowledgements

All authors were supported in part by Sunday Group, Inc. as part of its Möbby project. In addition to that the authors were supported by the following grants. Yun Lu was also supported by NSERC (RGPIN-03642-2023). Rafail Ostrovsky is also supported by NSF grants CNS-2246355, CCF-2220450, US-Israel BSF grant 2022370. Vassilis Zikas was also supported by NSF Award #2448339. Work done in part while this author was at Purdue University.

References

- AAMM24. Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and Subhra Mazumdar. Securing lightning channels against rational miners. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 393–407. ACM, 2024.
- ACKZ20. Aydin Abadi, Michele Ciampi, Aggelos Kiayias, and Vassilis Zikas. Timed signatures and zero-knowledge proofs - timestamping in the blockchain era -. In Mauro Conti, Jianying Zhou, Emiliano Casalicchio, and Angelo Spognardi, editors, *Applied Cryptography and Network Security - 18th International Conference, ACNS 2020, Rome, Italy, October 19-22, 2020, Proceedings, Part I*, volume 12146 of *Lecture Notes in Computer Science*, pages 335–354. Springer, 2020.
- AKMZ12. Joël Alwen, Jonathan Katz, Ueli Maurer, and Vassilis Zikas. Collusion-preserving computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 124–143. Springer, Berlin, Heidelberg, August 2012.
- alg. Instant finality - what makes algorand stand among blockchains. <https://developer.algorand.org/solutions/avm-evm-instant-finality/>.
- AST24. Zeta Avarikioti, Stefan Schmid, and Samarth Tiwari. Musketeer: Incentive-compatible rebalancing for payment channel networks. In Rainer Böhme and Lucianna Kiffer, editors, *6th Conference on Advances in Financial Technologies, AFT 2024, September 23-25, 2024, Vienna, Austria*, volume 316 of *LIPIcs*, pages 13:1–13:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- ASW97. N. Asokan, Matthias Schunter, and Michael Waidner. Optimistic protocols for fair exchange. In Richard Graveman, Philippe A. Janson, Clifford Neuman, and Li Gong, editors, *ACM CCS 97*, pages 7–17. ACM Press, April 1997.
- ASW98. N. Asokan, Victor Shoup, and Michael Waidner. Asynchronous protocols for optimistic fair exchange. In *1998 IEEE Symposium on Security and Privacy*, pages 86–99. IEEE Computer Society Press, 1998.
- BFK17. Alex Biryukov, Daniel Feher, and Dmitry Khovratovich. Guru: Universal reputation module for distributed consensus protocols. Cryptology ePrint Archive, Report 2017/671, 2017.
- BGK⁺24. Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Consensus redux: Distributed ledgers in the face of adversarial supremacy. In *CSF 2024 Computer Security Foundations Symposium*, pages 143–158. IEEE Computer Society Press, July 2024.
- BGM⁺18. Christian Badertscher, Juan A. Garay, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. But why does it work? A rational protocol design treatment of bitcoin. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 34–65. Springer, Cham, April / May 2018.
- BHMN21. Iddo Bentov, Pavel Hubáček, Tal Moran, and Asaf Nadler. Tortoise and hares consensus: The mesh-cash framework for incentive-compatible, scalable cryptocurrencies. In Shlomi Dolev, Oded Margalit, Benny Pinkas, and Alexander Schwarzmann, editors, *Cyber Security Cryptography and Machine Learning*, pages 114–127. Cham, 2021. Springer International Publishing.
- BK14. Iddo Bentov and Ranjit Kumaresan. How to use bitcoin to design fair protocols. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 421–439. Springer, Berlin, Heidelberg, August 2014.
- BLZ21. Christian Badertscher, Yun Lu, and Vassilis Zikas. A rational protocol treatment of 51% attacks. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 3–32, Virtual Event, August 2021. Springer, Cham.
- Can03. Ran Canetti. Universally composable signatures, certification and authentication. Cryptology ePrint Archive, Report 2003/239, 2003.
- CKKZ20. Michele Ciampi, Nikos Karayannidis, Aggelos Kiayias, and Dionysis Zindros. Updatable blockchains. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider, editors, *ESORICS 2020, Part II*, volume 12309 of *LNCS*, pages 590–609. Springer, Cham, September 2020.
- CLZ22. Michele Ciampi, Yun Lu, and Vassilis Zikas. Collusion-preserving computation without a mediator. In *CSF 2022 Computer Security Foundations Symposium*, pages 211–226. IEEE Computer Society Press, August 2022.
- CMSZ22. Christian Cachin, Jovana Micic, Nathalie Steinhauer, and Luca Zanolini. Quick order fairness. In Ittay Eyal and Juan A. Garay, editors, *FC 2022*, volume 13411 of *LNCS*, pages 316–333. Springer, Cham, May 2022.
- DGKR18. Bernardo David, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 66–98. Springer, Cham, April / May 2018.
- ETC. Ethereum classic hit by third 51 percent attack in a month. <https://www.coindesk.com/markets/2020/08/29/ethereum-classic-hit-by-third-51-attack-in-a-month/>.

- Eth. Proof-of-stake (pos). <https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/>.
- GCN⁺25. Tiantian Gong, Gustavo Franco Camilo, Kartik Nayak, Andrew Lewis-Pye, and Aniket Kate. Recover from excessive faults in partially-synchronous bft smr. *Cryptology ePrint Archive*, 2025.
- GHM⁺17. Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th symposium on operating systems principles*, pages 51–68, 2017.
- GKL15. Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 281–310. Springer, Berlin, Heidelberg, April 2015.
- GKM⁺13. Juan A. Garay, Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. Rational protocol design: Cryptography against incentive-driven adversaries. In *54th FOCS*, pages 648–657. IEEE Computer Society Press, October 2013.
- KDL⁺23. Mahimna Kelkar, Soubhik Deb, Sishan Long, Ari Juels, and Sreeram Kannan. Themis: Fast, strong order-fairness in byzantine consensus. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 475–489. ACM Press, November 2023.
- KK21. Dimitris Karakostas and Aggelos Kiayias. Securing proof-of-work ledgers via checkpointing. In *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–5, 2021.
- KOZ20. Leonard Kleinrock, Rafail Ostrovsky, and Vassilis Zikas. Proof-of-reputation blockchain with nakamoto fallback. In Karthikeyan Bhargavan, Elisabeth Oswald, and Manoj Prabhakaran, editors, *INDOCRYPT 2020*, volume 12578 of *LNCS*, pages 16–38. Springer, Cham, December 2020.
- KRDO17. Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. Ouroboros: A provably secure proof-of-stake blockchain protocol. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 357–388. Springer, Cham, August 2017.
- KZGJ20. Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. Order-fairness for byzantine consensus. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 451–480. Springer, Cham, August 2020.
- KZZ16. Aggelos Kiayias, Hong-Sheng Zhou, and Vassilis Zikas. Fair and robust multi-party computation using a global transaction ledger. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 705–734. Springer, Berlin, Heidelberg, May 2016.
- LJ23. Mario Larangeira and Maxim Jourenko. Maravedí: A secure and practical protocol to trade risk for instantaneous finality. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 285–313. Springer, Cham, July 2023.
- LPNRZ25. Andrew Lewis-Pye, Joachim Neu, Tim Roughgarden, and Luca Zanolini. Accountable liveness. *arXiv preprint arXiv:2504.12218*, 2025.
- LPR25. Andrew Lewis-Pye and Tim Roughgarden. Beyond optimal fault tolerance. *arXiv preprint arXiv:2501.06044*, 2025.
- MBB⁺19. Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. Sprites and state channels: Payment networks that go faster than lightning. In Ian Goldberg and Tyler Moore, editors, *FC 2019*, volume 11598 of *LNCS*, pages 508–526. Springer, Cham, February 2019.
- MMNT19. Bernardo Magri, Christian Matt, Jesper Buus Nielsen, and Daniel Tschudi. Afgjort: A partially synchronous finality layer for blockchains. *Cryptology ePrint Archive*, Report 2019/504, 2019.
- NTT22. Joachim Neu, Ertem Nusret Tas, and David Tse. The availability-accountability dilemma and its resolution via accountability gadgets. In *International Conference on Financial Cryptography and Data Security*, pages 541–559. Springer, 2022.
- NTT24. Joachim Neu, Ertem Nusret Tas, and David Tse. Short paper: Accountable safety implies finality. In *International Conference on Financial Cryptography and Data Security*, pages 41–50. Springer, 2024.
- pay. Payment channels - bitcoin wiki. https://en.bitcoin.it/wiki/Payment_channels.
- PS18. Rafael Pass and Elaine Shi. Thunderella: Blockchains with optimistic instant confirmation. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 3–33. Springer, Cham, April / May 2018.
- RAKM23. Sophie Rain, Georgia Avarikioti, Laura Kovács, and Matteo Maffei. Towards a game-theoretic security analysis of off-chain protocols. In *36th IEEE Computer Security Foundations Symposium, CSF 2023, Dubrovnik, Croatia, July 10-14, 2023*, pages 107–122. IEEE, 2023.
- RKK⁺22. Ranvir Rana, Dimitris Karakostas, Sreeram Kannan, Aggelos Kiayias, and Pramod Viswanath. Optimal bootstrapping of pow blockchains. In *Proceedings of the Twenty-Third International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, pages 231–240, 2022.
- Sat08. Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
- SWK⁺23. Peiyao Sheng, Xuechao Wang, Sreeram Kannan, Kartik Nayak, and Pramod Viswanath. TrustBoost: Boosting trust among interoperable blockchains. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 1571–1584. ACM Press, November 2023.

- SWKV21. Suryanarayana Sankagiri, Xuechao Wang, Sreeram Kannan, and Pramod Viswanath. Blockchain CAP theorem allows user-dependent adaptivity and finality. In Nikita Borisov and Claudia Díaz, editors, *FC 2021, Part II*, volume 12675 of *LNCS*, pages 84–103. Springer, Berlin, Heidelberg, March 2021.
- THFY23. Ertem Nusret Tas, Runchao Han, David Tse, and Mingchao Yu. Interchain timestamping for mesh security. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 1585–1599. ACM Press, November 2023.
- TTG⁺23. Ertem Nusret Tas, David Tse, Fangyu Gai, Sreeram Kannan, Mohammad Ali Maddah-Ali, and Fisher Yu. Bitcoin-enhanced proof-of-stake security: Possibilities and impossibilities. In *2023 IEEE Symposium on Security and Privacy*, pages 126–145. IEEE Computer Society Press, May 2023.
- Ver. Verge’s blockchain attacks are worth a sober second look. <https://www.coindesk.com/markets/2018/06/05/verges-blockchain-attacks-are-worth-a-sober-second-look/>.

A Additional Preliminaries

A.1 Signatures

Definition A.1 (Signature scheme [Can03]). A triple of PPT algorithms $(\text{Kgen}, \text{Sign}, \text{Ver})$ is called a signature scheme if it satisfies the following properties.

Completeness: For every pair $(s, v) \xleftarrow{\$} \text{Kgen}(1^\lambda)$, and every $m \in \{0, 1\}^\lambda$, we have that $\Pr[\text{Ver}(v, m, \text{Sign}(s, m)) = 0] < \nu(\lambda)$.

Consistency (non-repudiation): For any m , the probability that $\text{Kgen}(1^\lambda)$ generates (s, v) and $\text{Ver}(v, m, \sigma)$ generates two different outputs in two independent invocations is smaller than $\nu(\lambda)$.

Unforgeability: For every PPT $\mathcal{A}$, there exists a negligible function ν , such that for all auxiliary input $z \in \{0, 1\}^*$ it holds that:

$$\Pr[(s, v) \xleftarrow{\$} \text{Kgen}(1^\lambda); (m, \sigma) \xleftarrow{\$} \mathcal{A}^{\text{Sign}(s, \cdot)}(z, v) \wedge \text{Ver}(v, m, \sigma) = 1 \wedge m \notin Q] < \nu(\lambda)$$

where Q denotes the set of messages whose signatures were requested by $\mathcal{A}$ from the oracle $\text{Sign}(s, \cdot)$.

B Blockchain Definitions Revisited

Here we slightly revisit the definition of liveness and safety. Formal definitions follow, and all of them are required to hold, except with negligible probability in the security parameter, against a *ppt* adversary.

Definition B.1 (Safety). A blockchain protocol that follows the abstraction of Fig. 4.3 satisfies safety with parameter co if for every slot $t \in [0, \dots, m]$, with $\text{ASP}(t) = 1$ and $\text{GOracle.corrupted} = 0$, then the following occurs: at any time slot $i \in [0, \dots, m]$, the local chains held by any two honest parties are consistent if they each ignore their co most recent blocks.

Definition B.2 (Instant Finality). A blockchain protocol that follows the abstraction of Fig. 4.3 satisfies instant finality if for every slot $t \in [0, \dots, m]$ with $\text{ASP}(t) = 1$ and $\text{GOracle.corrupted} = 0$ the following occurs: in the slot interval $[0, \dots, m]$, once a block becomes part of any honest party’s chain, it is not reverted.

Definition B.3 (Block Liveness). A blockchain protocol that follows the abstraction of Fig 4.3 satisfies block liveness with parameter liv if for every slot $t \in [0, \dots, m]$ with $\text{ASP}(t) = 1$ and $\text{GOracle.corrupted} = 0$ the following occurs: in the interval $[0, \dots, m - \text{liv}]$ a new block is added to each local chain held by any honest party within liv slots.

Definition B.4 (Transaction Liveness). A blockchain protocol that follows the abstraction of Fig. 4.3 satisfies transaction liveness with parameter liv if for every $t \in [0, \dots, m]$ with $\text{ASP}(t) = 1$ and $\text{GOracle.corrupted} = 0$ the following occurs: if a transaction is submitted by an honest party via the protocol TxSubmit in the slot τ_0 , then the transaction is added to each local chain held by any honest party within liv slots conditioned on $\tau_0 + \text{liv}, \tau_0 \in [0, \dots, m - \text{liv}]$.

B.1 More on reputation systems

Definition B.5 (Reputation System [KOZ20]). A (static) reputation system Rep for $m = O(\lambda)$ reputation parties $P = \{P_1, \dots, P_m\}$ is a probability distribution over binary vectors of length m , called reputation vectors. If $h = (h_1, \dots, h_m)$ is a reputation vector, then $h_i = 1$ indicates P_i is honest, and $h_i = 0$ indicates P_i is dishonest. We also define a dynamic reputation system (where reputations can change), where $\text{Rep} = \{\text{Rep}_\rho\}_{\rho \in \mathbb{N}}$ is a sequence of static reputation systems.

Definition B.6 (Feasibility [KOZ20]). Let Rep be a reputation system for set of reputation parties P . We say that Rep is ϵ -feasible for an adversary $\mathcal{A}$ if there exists a PPT algorithm A which, if with overwhelming probability, outputs a set of parties such that at most $1/2 - \epsilon$ fraction of them is corrupted by $\mathcal{A}$.

B.2 Security Properties of Our Hybrid Protocol

Theorem B.1 ((Thm. 6.1) Safe PoR, Compromised PoS). If $\text{ASP}^{\text{PoR}}(t) = 1$ for every $t \in [0, \dots, m]$, then at any time slot $i \in [0, \dots, m]$ Π enjoys safety, liveness, and transaction liveness of the underlying PoR chain.

Proof. In our protocol Π , when ASP^{PoR} holds, parties simply follow the *normal mode*. In this, the parties run the PoR protocol as prescribed (i.e., no forks are happening in the interval $[0, \dots, m]$ because ASP^{PoR} holds). The only other way that safety can be broken is if some honest parties go into recovery mode, whereas some other honest parties do not. There are two ways to go into recovery mode, 1) Some honest parties think that there is a fork, hence some of them go into recovery mode, and some others do not. 2) Some parties detect a liveness violation, and some others do not.

Consider the first case, and assume by contradiction that parties go into recovery mode because a fork has been detected. In this case, for an honest party to go into recovery mode, they must see a valid checkpoint on the PoS chain, which is inconsistent with the local view they have. More formally, the PoS chain contains $\text{tx} = (\text{CHECK-POINT}, h^*, k, \text{cert}^*)$, such that $\text{certify}^{\text{PoR}}(\text{status}_{k-1}^{\text{PoR}}, H(B_{k-1}^{\text{PoR}}), h^*), k, \text{cert}^*) = 1$, and $H(B_k^{\text{PoR}}) \neq h^*$, where $\text{status}_{k-1}^{\text{PoR}}$ represent the reputation state at the k -th slot, $H(B_{k-1}^{\text{PoR}})$ (resp. $H(B_k^{\text{PoR}})$) is the hash of the PoR block appearing in the local view of the party at slot $k-1$ (resp. k). However, if this happens during a time frame where $\text{ASP}^{\text{PoR}}(t) = 1$, then this would contradict Property 5.2 (i.e., certificate unforgeability) of the PoR chain.

Assume now by contradiction that some honest parties detect a liveness violation and go into recovery mode. The only reason why honest parties go into recovery mode is if they receive a PoR block with a delay. Given that the PoR is assumed to be secure, then all honest parties will receive the PoR block and never trigger the liveness-violation mechanism.

We finally note that a compromised secondary chain may force the inclusion of the transaction (LIVENESS-VIOLATION). However, this transaction is ignored by any honest party that receives a PoR block in the expected slot. Given that ASP^{PoR} holds, we are guaranteed that all the honest parties will ignore (LIVENESS-VIOLATION) as all of them will receive a valid PoR block in every slot.

We can use the same arguments to prove that the liveness of Π corresponds to the liveness of the PoR chain. Indeed, honest parties will ever switch from normal to recovery mode due to the same arguments.

Theorem B.2 ((Thm. 6.2) Safety: Compromised PoR, Safe PoS). If $\text{ASP}^{\text{PoS}}(t) = 1$ for every $t \in [0, \dots, m]$ then Π enjoys safety: at any time slot $i \in [0, \dots, m]$, the local chains held by any two honest parties are consistent if they each ignore the blocks generated in the latest $D = \max\{2\text{liv}^{\text{PoS}}, \text{co}^{\text{PoS}}\}$ PoS slots.

Proof. Let E_i^τ be the event in which the honest party p_i , in some slot τ , detects a conflict (i.e., the hash of the i -th checkpointed block on the secondary chain does not correspond with the hash of the i -th block of the local PoR chain of the party). Formally, E_i^τ is the event where on the PoS chain there is a transaction $\text{tx} = (\text{CHECK-POINT}, h, k, \text{cert})$ (or $\text{tx} = (\text{COMPLAINT}, h, \text{cert}, k)$), such that $h \neq h^*$, where h^* is the hash of the k -th block added to the local chain of p_i .

Claim. If there exists an honest party p_i for which $P[E_i^\tau]$ is non-negligible, then for each honest party p_j there exists a slot τ^j such that $P[E_j^{\tau^j}]$ is non-negligible with $\tau, \tau^j \in [0, \dots, m]$ and $|\tau - \tau^j| \leq \Delta$.

Proof. If the event E_i^T happens, then our protocol mandates p_i to issue a transaction $\text{tx} = (\text{COMPLAINT}, h, \text{cert}, k)$, unless this transaction is already on the PoS chain. Given that by assumption ASP^{PoS} holds, then after Δ PoS slots, tx will appear on the view of all the honest parties. If this does not occur, then either we can contradict the security of the PoS or the collision resistance of the hash function.

The case where also $\text{ASP}^{\text{PoR}}(t) = 1$ for every $t \in [0, \dots, m]$ follows immediately from Theorem 6.1. Hence, in the rest of the proof we consider the case where $\text{ASP}^{\text{PoR}}(k) = 0$ for some $k \geq 0$.¹⁴ In the first part of the proof, we assume that only the safety of the PoR chain is compromised, but the block liveness property holds.

The claim we have proven guarantees that all the honest parties, in the event of a safety violation, would detect such a violation. Our protocol mandates that all parties that detect such a violation enter recovery mode, and ignore the transactions generated in the latest 2liv^{PoS} PoS slots. Indeed, note that if a fork happens and a party notices that, then this party needs liv^{PoS} to issue the complaint. Moreover, all the transactions generated between the latest undisputed checkpoint and the disputed checkpoint must be discarded, as, in the worst case, they could all be part of a fork. These transactions are represented by the transaction generated starting right after the latest undisputed checkpointed block, and the block containing the complaint (i.e., when a checkpoint is disputed). Hence, the maximum number of PoS slots that incur between the last undisputed checkpoint and the detection of the fork (i.e., the complaint is part of the view of all honest parties) is 2liv^{PoS} PoS slots. After parties go into recovery mode, they just run the PoS chain, adopting the safety rule of the PoS chain (hence, cutting the blocks generated in the latest co^{PoS} slots).

In the case of a liveness violation, either there exists at least one honest party that receives the PoR blocks correctly (hence, they can diffuse them to all the other parties within the next slot), or parties need to rely on the PoS chain to ask for missing blocks. In the worst case, parties will go into recovery mode and exclusively run the PoS chain. In both cases, the safety of the overall protocol becomes co^{PoS} .

Theorem B.3 ((Thm. 6.3) Liveness: Compromised PoR, Safe PoS). *If $\text{ASP}^{\text{PoS}}(t) = 1$ for all $t \in \mathbb{N}$, then a transaction tx submitted by an honest party is included in the honest parties' local chains within $3\text{liv}^{\text{PoS}} + \text{co}^{\text{PoS}}$ PoS slots.*

Proof. We start by considering the case where only the liveness of the PoS is compromised. In this case, two main issues can occur: 1) some honest parties keep receiving blocks at every PoR slot while others do not, and 2) none of the honest parties receive the PoR blocks in a specific slot.

If case 1) happens, then all honest parties are guaranteed to receive the block of slot t in slot $t + 1$ (i.e., with a one-slot delay). Indeed, we recall that our protocol, when in normal mode, mandates the honest parties to diffuse the block (if any) they generate in a given slot.

If case 2) happens instead, we note that each honest party that does not receive a block in a slot starts querying the oracle $\mathcal{O}^{\text{liv}}$ of the PoS chain with the input (LIVENESS-VIOLATION). The property of Δ -External Signaling (Property 5.1) guarantees that if all honest parties query the oracle and do not receive a PoR block, then the transaction (ORACLE, LIVENESS-VIOLATION) appears on the PoS chain, denoting that there is a liveness violation. At this point, the parties can start requesting a missing PoR block on the PoS chain by issuing the transaction (BLOCK-REQUEST, i) (for $i \in \mathbb{N}$). If no transaction (BLOCK-REQUEST-RESPONSE, i, B) is issued in response to the block-request, then honest parties agree that there is a liveness violation, will go into recovery mode, and start accepting new transactions only on the PoS chain.

We now observe that if parties are in recovery mode, then a transaction can take up to liv^{PoS} PoS slots to be propagated to all the parties. If, instead, there are honest parties that manage to gather PoR blocks by issuing block-request transactions (and receiving valid block-request-response transactions), then these parties will receive an every block only within 2liv^{PoS} PoS slots (liv^{PoS} to propagate the block-request and liv^{PoS} to propagate the block-request-response).

On top of these considerations, we also need to consider the situation where the PoR forks, thus adding an additional delay to honest transactions that could potentially be part of a fork as discussed in the proof of Theorem 6.1. In this case, a transaction may take up to 3liv^{PoS} to be accepted. Indeed, as discussed in the proof of Theorem 6.1. 2liv^{PoS} PoS slots may be needed to detect a fork, go into recovery

¹⁴ This holds assuming that $\text{co}^{\text{PoS}} \geq \text{co}^{\text{PoR}}$ and that a PoS slot is at least as long as a PoR slot.

mode (hence discarding the honest transactions that are part of the fork generated in the latest 2liv^{PoS} PoS slots), after that, the honest party reissues the transaction, which will take up to liv^{PoS} slots to be accepted.

We also need to consider that the parties, before trying to go into normal mode again, to wait co^{PoS} slots without accepting transactions. This is to make sure that parties have an agreement on the new genesis block of the PoR (generated using a deep enough block in the PoS chain). However, after that, parties go into normal mode again, a fork may be immediately detected, and parties would need to go into recovery mode again. This means that an additional 3liv^{PoS} slots would be needed in order for an honest transaction to be accepted (in recovery mode). Hence, in the very worst case, a transaction may take up to $3\text{liv}^{\text{PoS}} + \text{co}^{\text{PoS}}$ PoS slots¹⁵.

Theorem B.4 ((Thm. 6.4) Recovery: Compromised PoR, Safe PoS). *If $\text{ASP}^{\text{PoR}}(t) = 1$ for every $t \geq t_1$ (for arbitrary $t_1 \in \mathbb{N}$) and $\forall k \text{ASP}^{\text{PoS}}(k) = 1$ and Π^{PoR} enjoys the property of seed recoverable security, then let τ be the time at which a transaction tx is submitted by an honest party. If $\tau \geq t_1 + \text{rectime} + 4\text{liv}^{\text{PoS}} + \text{co}^{\text{PoS}}$ then tx is included in the honest parties' local chains within liv^{PoR} PoR slots. Moreover, after the time slot τ , the safety of Π holds with respect to the parameter co^{PoR}*

Proof. As we have argued in Theorem 6.3 and Theorem 6.1, when $\text{ASP}^{\text{PoR}}(t)$ does not hold, a liveness or a safety violation can slow down the protocol. In the worst case, the protocol goes into recovery mode, where liveness and safety degrade as formally stated in Theorem 6.3 and Theorem 6.1. We want to argue that at any point ASP^{PoR} starts to hold again, then after τ (defined as in the theorem statement), the performance of the protocol in terms of liveness and safety matches that of the PoR chain.

In the worst case, when ASP^{PoR} does not hold, the protocol goes into recovery mode, and after rectime , tries to bootstrap the PoR protocol using a freshly generated genesis block. By assumption, we know that there exists a time slot t , after which ASP^{PoR} holds. Moreover, we know that in the worst case (i.e., when there is a safety or liveness violation), the protocol will try to reboot the PoR chain. Again, in the worst case, ASP^{PoR} could start to hold again, right after a fork is created or a safety violation is detected (hence, the procedure to go into recovery mode is triggered). Entering into recovery mode takes 2liv^{PoS} , and after that, the parties wait for rectime and agree on a genesis block G , whose seed is created by hashing the PoS block generated at the end of rectime . Parties make sure to agree on the genesis block by waiting for additional co^{PoS} PoS slots. Given that by assumption ASP^{PoR} holds, then parties should be able to start running again the PoR chain using this new genesis block G .

In order for the above to hold true, we need to argue that G is created accordingly to the requirement of the PoR chain. In particular, we need to guarantee that the mechanism we use to generate G realizes the ideal functionality $\text{GOracle}^{\text{PoR}}$.

This part of the proof proceeds as follows. The simulator controls the random oracle, and upon receiving a query $H(\text{GENESIS}, B || \text{dspor})$, send the message `gen_genesis` to GOracle , and upon receiving the answer `seed`, programs the output of the random oracle to output `seed` when receiving the input $H(\text{GENESIS}, B || \text{dspor})$. Let B^* be the block added to the PoS chain at the end of rectime . After co^{PoS} slots (i.e., wait for B^* to become a stable part of the PoS chain) the simulator computes $H(\text{GENESIS}, B^* || \text{dspor})$ thus obtaining `seed*`, and send the message `(set_genesis, seed*)` to GOracle . Note that if the adversary has already queried the random oracle with $H(\text{GENESIS}, B^* || \text{dspor})$ then our simulation strategy would have already send the command `gen_genesis`, thus obtaining `seed*`, that is used to program the output of the random oracle on the input $H(\text{GENESIS}, B^* || \text{dspor})$.

To argue that the above simulation works, we make the following final observations. The simulator will send the command `(set_genesis, seed*)` to GOracle before delay slots. This is because the liveness of the PoS chain guarantees that new blocks will be added, and more importantly, the chain quality of the PoS guarantees that the real-world adversary cannot predict the full content of B^* more than 2cq^{PoS} in advance. We recall that this holds due to the unpredictability property of the PoS chain. Moreover, note that the GOracle we are realizing is parametrized by $\text{delay} = 2\text{cq}^{\text{PoS}}$.

On a final note, we stress that in the above proof, we have assumed that a liveness or a safety violation is detected. However, it could very well be the case that parties never need to go into recovery mode as no liveness or safety violation is detected. However, we still need to argue that after the assumption

¹⁵ Recall that we assume that $\text{rectime} \geq 5\text{liv}^{\text{PoS}} + 2\text{co}^{\text{PoS}}$, hence, all the transactions of the honest parties issued right after being in recovery mode are guaranteed to be accepted. We note that, in this proof, following the related literature, we work in an idealized world where blocks have unlimited space.

is restored, the PoR chain regains its security completely. In particular, in our case, this is guaranteed by the property of *seed recoverable security* held by the PoR chain. We recall that this guarantees that the seed generation cannot be biased by an adversary once the assumption that underlines the protocol holds again.

Theorem B.5 ((Thm. 6.5) Payment Finality). *If an honest party p accepts a safe transaction of value v in the slot τ and $\forall k \text{ASP}^{\text{PoS}}(k) = 1$, and $\exists t$ such that $\text{ASP}^{\text{PoR}}(t) = 0$ then the stake represented by the chain of all honest parties in the slot $(t_1 + 4\text{liv}^{\text{PoS}})$ will reflect the fact that the balance of p has increased by v .*

Proof. This proof follows from the proofs of the previous theorems, and from the description of how Π works. Indeed, a payment can be confirmed in the next slot, when ASP^{PoR} is assumed to hold. If ASP^{PoR} does not hold, but a party does accept a safe transaction, this means that ASP^{PoS} holds, and moreover, the *pledge* transaction was correctly checkpointed and undisputed. Hence, if a fork happens, due to the previous theorems, the protocol will go into recovery mode, and new transactions will be accepted at the speed of the PoS chain. In particular, this means that the receiver of the transaction will be able to complain on the PoS via the transaction `PAYBACK`. In conclusion, in the worst case, a delay of 4liv^{PoS} will occur because (as argued in Theorem 6.3) the standard payment transaction may be part of a fork, hence, it will be discarded and the system would need to go into recovery mode. After that, the honest party can create and issue the transaction `(PAYBACK, ·)`, which will be accepted by the PoS chain within additional liv^{PoS} PoS slots.

C Proofs and Further Discussions for Sec. 7

Depositing Collaterals for Fast Payments. Here, we describe the simpler case (compared to introducing a treasury) for fast payments, where the collateral is deposited by the sender p_i using their own funds. To deposit a collateral (before a pledge can be made to another party), a party p_i can submit the following transaction.

- $\text{tx} = (\text{DEPOSIT-COLLATERAL}, \text{addr}_i, v, T)$: A party p_i deposits and locks up v coins, with expiry T . This transaction is valid if indeed p_i has v coins and current slot is before $T - \Delta$.
- $\text{On}(\text{CheckDeposit}, \text{chain}, p, \Delta)$: Returns the amount currently deposited by p , according to PoR chain `chain`. This returns the total amount deposited by p , minus the total amount paid out from these deposits.

A deposit is accepted (i.e., a `PLEDGE` transaction is valid from a party p_i) once tx is checkpointed on the PoS chain. Notably, unlike other Layer 2/off-chain solutions like the payment channel, as long as the transaction sender (the person pledging this collateral) does not double spend, this deposit is not used—so this deposit does not need to be updated until the expiry of this transaction.

Remark: Counting stake For simplicity, our proofs will assume the treasury allocation counts towards stake for the PoS (i.e., the stake calculation also computes `(CheckAllocation, chain, p, Δ)` for party p). In practical scenarios this usually does not make a big difference as treasury allocation is generally small in comparison to the number of coins held by each party, but this solves a subtle technical issue. That is, if treasury allocation counts towards stake, then double spending *cannot increase* stake for the double spender. Thus, when analyzing the incentives against double spending (as we do later), we do not need to consider the increase in PoS block rewards for the double spender. Note that, since honest parties only perform treasury allocation calculation on the settled part of the PoR chain (Δ slots before current slot), all honest PoS parties will agree on the stake stemming from the treasury allocation (since this is checkpointed on the PoS), or parties will have detected a fork in the PoR (in which case, recovery would be initiated).

Proof (Proof: Lem. 7.1). Suppose p_i pledges to an honest party p_j , and p_j accepts (unconfirmed/not yet checkpointed) transaction(s) from p_i at some t_1 . This means the amount needed to pay back for these transactions is at most `(CheckAllocation, chain1, pi, Δ)`, where `chain1` is the PoR chain of p_j at t_1 .

Now, suppose p_i double spends and p_j needs to be paid back. This is done during recovery phase, where the honest p_j will submit a `PAYBACK` transaction (as well as all unconfirmed transactions sent by p_i). By allocation persistence and agreement, any honest party with some chain

chain_2 who checks $(\text{CheckAllocation}, \text{chain}_2, p_i, \Delta)$ at this point will see a value at least as big as $(\text{CheckAllocation}, \text{chain}_1, p_i, \Delta)$, and accept the PAYBACK. The treasury is always sufficient, thus there are sufficient coins to pay back the double-spend. Then payment finality follows from the rest of Thm. 6.5 proof.

Proof (Proof: Thm. 7.1). We start with the first four properties.

- *Allocation agreement:* The difficulty here is that two honest parties may hold conflicting PoR chains. However, by assumption of the hybrid protocol, honest parties agree on the current slot number t , and the PoR chain up to slot $t - \Delta$ (recall $\Delta = 2 \cdot \text{liv}^{\text{PoS}}$ is the number of slots for a PoR block to finalize in the hybrid protocol). Thus, since **TreasuryAllocation** computes the allocated amount using slot $t - \Delta$, honest parties will compute the same result from **TreasuryAllocation** for the same party p .
- *Allocation persistence* holds since if between slots t_1, t_2 : (1) if there is no recovery phase, then neither is treasury used (so treasury has not been decreased), nor do parties' reputations decrease (so **TreasuryAllocation** returns the same or higher result for the same party); (2) if there is recovery phase (but recovery phase has not ended), all honest parties halt their PoR chain while they perform recovery in PoS. (until the end of recovery phase)—this means that similarly, **TreasuryAllocation** returns the same or higher result.
- *Fair allocation* holds since **TreasuryAllocation**'s output only depends on party p 's reputation compared to other parties.
- *Treasury is always sufficient:* This holds since PAYBACK only pays for double-spent transactions from sender p_i to receiver p_j up to the amount returned by **CheckAllocation** during recovery phase.

For *no incentive to misbehave for parameter Δt* : Let p_a and p_h have the same reputation r and stake except p_h is honest and p_a is found to misbehave (and subsequently has their reputation zeroed at time t_a). We want to show:

$$u_{t, \Delta t}(p_h) > u_{t, \Delta t}(p_a)$$

Since p_h doesn't fork:

$$u_{t, \Delta t}(p_h) = \sum_{t \in [t_a, t_a + \Delta t]} \text{rew}_{\text{PoR}} \cdot \Pr(I_{t, r_t}^{\text{PoR}}) + \sum_{t \in [t_a, t_a + \Delta t]} \text{rew}_{\text{PoS}} \cdot \Pr(I_{t, s_t}^{\text{PoS}})$$

Moreover, since p_a 's reputation is zeroed, and by allocation persistence p_a 's profit from double spending is at most the treasury allocation $v = (\text{CheckAllocation}, \text{chain}, p_a, \Delta)$ for the honest PoR chain at slot t_a :

$$u_{t, \Delta t}(p_a) \leq \sum_{t \in [t_a, t_a + \Delta t]} \text{rew}_{\text{PoS}} \cdot \Pr(I_{t, s_t}^{\text{PoS}}) + v$$

Now, since we count treasury allocation as stake, p_a and p_h have the same amount of stake even after the double spending, so we have

$$\begin{aligned} & u_{t, \Delta t}(p_h) - u_{t, \Delta t}(p_a) \\ &= \sum_{t \in [t_a, t_a + \Delta t]} \text{rew}_{\text{PoR}} \cdot \Pr(I_{t, r_t}^{\text{PoR}}) - v \end{aligned}$$

Since we assume no new parties are introduced within Δt of recovery, and p_h 's reputation does not decrease during this time interval, p_h 's probability of creating a PoR block cannot decrease from $\Pr(I_{t_a, r}^{\text{PoR}})$. So the above is:

$$\geq \Delta t \cdot \text{rew}_{\text{PoR}} \cdot \Pr(I_{t_a, r}^{\text{PoR}}) - v$$

Let $T = (\text{CheckTreasury}, \text{chain})$ (note that while we're in recovery phase, the PoR chain has halted, so the treasury does not change during this time). By the way we defined **TreasuryAllocation**, $v \leq T \cdot \frac{\Pr(I_{t_a, r}^{\text{PoR}})}{m}$, where m is the number of parties chosen by the PoR lottery. So the above is:

$$\begin{aligned} & \geq \Delta t \cdot \text{rew}_{\text{PoR}} \cdot \Pr(I_{t_a, r}^{\text{PoR}}) - T \cdot \frac{\Pr(I_{t_a, r}^{\text{PoR}})}{m} \\ &= \Pr(I_{t_a, r}^{\text{PoR}}) \left(\Delta t \cdot \text{rew}_{\text{PoR}} - \frac{T}{m} \right) \end{aligned}$$

By assumption, the treasury at any point is $< m \cdot \Delta t \cdot \text{rew}_{\text{PoR}}$, which makes our expression above be > 0 , proving our desiderata property.

C.1 Further Improvements to the Incentives Structure

For simplicity of presentation we proposed in this section a basic incentives structure where the utility function includes block rewards (for participating in the underlying PoR or PoS chains), as well as the amount of system funds (treasury) which can be distributed to parties according to reputation. There are several future directions for improvement. One direction is to specify the exact mechanism/algorithm for increasing a party's reputation when they demonstrate honest behaviour (in our theorems, we only assume that reputation does not increase too fast). We can also consider different kinds of block rewards for parties with different roles in the consensus algorithm. Lastly, when considering Decentralized Finance applications, the ability to (re)order transactions (captured by the term Maximal Extractable Value, MEV) is also another source of utility for a party. An interesting future direction is to consider MEVs during our protocol's recovery phase, such as by introducing a data-independent ordering of transactions, e.g., [KZGJ20, KDL⁺23, CMSZ22].

Adding fees for posting on PoS. To prevent an adversary from submitting invalid transactions on the PoS chain (abusing the liveness violation detection mechanism described above), thus flooding the network, we could ask every party issuing a transaction on the PoS chain to pay a fee. However, the PoS chain has no way to check if a party has enough money to afford the fee (again, the PoS does not see previous transactions hence it cannot check if the party can afford the fee). To solve this problem, we just need every party to have some locked money in the PoS chain, that will be used to pay for a transaction posted on the PoS (i.e., the locked money can be used only to pay for a specific transaction, that in this case, it will be a transaction posted on the PoS). This transaction will be bundled with another transaction that locks again the same amount of money. This guarantees that a new transaction can be posted on the PoS chain if needed. Of course, parties may decide not to do that, but this means that they will not have the chance to submit a transaction on the PoS in the event the PoS is censorship them. We note that such a special transaction will never need to be issued by an honest party unless the PoR chain is compromised due to a liveness violation.